

Predicting Property Prices in Perth Using Big Data Analytics: A PySpark and Tableau Approach

Benjamin IBOK

15323543

Department of

Data Science and

Computational Intelligence

Coventry University

United Kingdom



Abstract

This work seeks to explore the world of Real Estate in an Australian City named Perth between 1988 to 2020, using Data Mining tools, and the application of Big Data concepts. The experiment uses linear regression and tree ensemble regression models like Linear Regression, Generalized Linear Regression (GLR), Extreme Gradient Boosting Regressor (XGBoost), and Random Forest Regressor, compares their performance on a real estate dataset using the Root Mean Squared Error (RMSE), R-Squared Score (R^2) and Mean Absolute Error (MAE) evaluation metrics. At the end of the experiment, an Ordinary Least Squares (OLS) Regression Result can be generated for any of the selected models. XGBoost emerged the best model with a R^2 score of 80.7% which is a relatively good fit, however one of the recommendations for model improvement included applying log transformation to the target variable to make the distribution look more normal, another recommendation would be to approach the problem using time series with hyperparameter tuned models.

Keywords: Regression, R Score, Ordinary Least Squares, XGBoost Regressor, Time Series Analysis, Data Mining, Mean Absolute Error, Random Forest Regressor, Tableau Dashboard, Feature Engineering, Data Preprocessing, Label Encoding, Generalized Linear Regression, PySpark, Residuals Plot.

Table of Contents

Abstract	-	-	-	-	-	-	-	-	-	-	2
Table of Contents	-	-	-	-	-	-	-	-	-	-	3
1.0 Introduction	-	-	-	-	-	-	-	-	-	-	5
2.0 Related Work	-	-	-	-	-	-	-	-	-	-	5
3.0 Description of Dataset	-	-	-	-	-	-	-	-	-	-	6
4.0 Machine Learning Methods	-	-	-	-	-	-	-	-	-	-	6
4.1 PySpark MLlib	-	-	-	-	-	-	-	-	-	-	6
4.2 Linear Regression	-	-	-	-	-	-	-	-	-	-	7
4.3 Generalized Linear Regression	-	-	-	-	-	-	-	-	-	-	7
4.4 Random Forest Regressor	-	-	-	-	-	-	-	-	-	-	7
4.5 XGBoost Regressor	-	-	-	-	-	-	-	-	-	-	8
4.6 Decision Tree Regressor	-	-	-	-	-	-	-	-	-	-	8
4.7 Time Series Plot	-	-	-	-	-	-	-	-	-	-	8
4.8 Tableau Overview	-	-	-	-	-	-	-	-	-	-	8
5.0 Explanation of the Experiment	-	-	-	-	-	-	-	-	-	-	9
5.1 Exploratory Data Analysis	-	-	-	-	-	-	-	-	-	-	10
5.2 Data Processing	-	-	-	-	-	-	-	-	-	-	11
5.3 Feature Engineering	-	-	-	-	-	-	-	-	-	-	11
5.4 Data Encoding / Label Encoding	-	-	-	-	-	-	-	-	-	-	12
5.5 Model Preparation	-	-	-	-	-	-	-	-	-	-	12
6.0 Tableau Visualization	-	-	-	-	-	-	-	-	-	-	12
7.0 Discussion of Results	-	-	-	-	-	-	-	-	-	-	13
7.1 RMSE	-	-	-	-	-	-	-	-	-	-	13
7.2 R^2	-	-	-	-	-	-	-	-	-	-	13
7.3 MAE	-	-	-	-	-	-	-	-	-	-	14
7.4 Prediction Scatter Plot	-	-	-	-	-	-	-	-	-	-	14
7.5 Residual Distribution	-	-	-	-	-	-	-	-	-	-	15
7.6 Residual Scatter Plots	-	-	-	-	-	-	-	-	-	-	15

7.7	Ordinary Least Squares (OLS) Results	-	-	-	-	-	-	-	16
8.0	Conclusion	-	-	-	-	-	-	-	17
9.0	Further Research	-	-	-	-	-	-	-	17
10.0	Social/Ethical Impact	-	-	-	-	-	-	-	17
11.0	References	-	-	-	-	-	-	-	17
	Appendix I (Links)	-	-	-	-	-	-	-	19
	Appendix II (Screenshots)	-	-	-	-	-	-	-	20
	Appendix III (Pasted Codes)	-	-	-	-	-	-	-	28

1.0 Introduction

Big data has revolutionized real estate by enabling data-driven decision-making, improving property valuation models, and enhancing market predictions (Wei et al., 2022). In this study, insights about the Real Estate marketplace in Perth is uncovered, releasing information which can equally be useful for both the Real Estate Agencies and Tenants alike, by integrating different tools in Machine Learning and Big Data. In Real Estate, House pricing is a complex and dynamic field which can be affected by many factors such as location, property size, number of bedrooms, or size of land (Ullah & Sepasgozar, 2020). For this reason, care must be taken, considering all these factors, to ensure accurate prediction of property prices as the outcome is crucial for investors, buyers, and policymakers to make informed decisions. Traditional methods of analysis often struggle with large datasets, making Big Data Analytics an essential approach to handling, processing, and deriving insights from property data (Iqbal et al., 2020).

Real estate pricing importance lies in their wide-ranging impact on individuals, the economy, and society. The study of traditional real estate prices has been guided mainly by the theory of hedonic prices (Rosen, Citation1974), which has been well accepted and recognized as viable and practicable (Oladunni & Sharma, 2016). However, using Big Data techniques like Linear Regression, Random Forest Regressor and the XGBoost Regressor models for house price prediction reduces the risk of over-lending on overvalued properties, because it identifies price fluctuations based on economic factors, demographics, or location.

2.0 Related Work

In a study by (Dharmaraj et al., 2024), PySpark, a big data processing framework with a built-in MLlib library, is used to predict real estate prices in metropolitan India. The researchers analysed large-scale housing data and identified the most effective algorithms using the MLlib library. Furthermore, the paper explores the use of PySpark MLlib to manage large-scale datasets and perform parallel processing, improving the accuracy and efficiency of real estate price prediction models. The approach is implemented on a dataset of real estate pricing factors from tier 1 cities in India, providing vital insights into urban real estate markets due to their economic and demographic significance. It incorporates various machine learning algorithms, including AdaBoost, Naïve Bayesian, C4.5, CRISP-DM, and sentiment analysis, to predict future real estate prices. The experiment yielded a performing model with the best being The Gradient-Boosted Tree Regression (GBTR or XGBoost) with a RMSE of 0.03216, MSE of 0.00103, MAE of 0.0085, and an R2 score of 0.7431.

Correspondingly, (Sharma et al., 2024) presented an optimal house price prediction using XGBoost, and the algorithm is used to predict house prices in Ames City, Iowa. The researchers used a housing dataset to compare various algorithms and identified key factors influencing housing costs. Then they formulated the problem as a regression task and conducted a comprehensive experimental comparison of ML

techniques to ascertain the most effective model that accurately predicts house prices, and the results also showed XGBoost as the best performing model in the experiment.

Subsequently, a study by (Zhao et al., 2024) proposed a novel approach to house price prediction from a multi-source data fusion perspective. The methodology involved analysing house characteristics and incorporating factors from diverse aspects, including amenities, traffic, and emotions. The approach is validated using a dataset of 28 550 real-world transactions in Beijing, China, providing a comprehensive analysis of the drivers influencing house prices, and in another study, (Zhang et al., 2024) explores house price prediction using textual description data, focusing on house asking/advertising price.

3.0 Description of Dataset

The dataset used in the experiment is a Kaggle dataset on House Prices Prediction for Perth, a city is Western Australia, from 1988 to 2020, It has 33,656 rows and 18 columns, 3 datatypes namely, 3 doubles, 8 integer columns and 7 strings in Table 1 below.

SN	Name	Description	Datatype
1	ADDRESS	Where the property is located	<i>string</i>
2	SUBURB	Suburb that property is located	<i>string</i>
3	BEDROOMS	Number of bedrooms available	<i>integer</i>
4	BATHROOMS	Number bathrooms per apartment	<i>integer</i>
5	GARAGE	Number of garages in each property	<i>string</i>
6	LAND_AREA	Land space of the property	<i>integer</i>
7	FLOOR_AREA	Overall floor space per property	<i>integer</i>
8	BUILD_YEAR	Year property was constructed	<i>string</i>
9	CBD_DIST	Distance to Central Business District	<i>integer</i>
10	NEAREST_STN	Name of nearest train station	<i>string</i>
11	NEAREST_STN_DIST	Proximity to the nearest train station	<i>integer</i>
12	DATE_SOLD	Date property was sold	<i>string</i>
13	POSTCODE	Property postcode	<i>integer</i>
14	LATITUDE	Location latitude	<i>double</i>
15	LONGITUDE	Location longitude	<i>double</i>
16	NEAREST_SCH	Nearest school name	<i>string</i>
17	NEAREST_SCH_DIST	Proximity to the nearest school	<i>double</i>
18	NEAREST_SCH_RANK	Nearest school ranking	<i>integer</i>

Table 1. Showing a description of the dataset features

4.0 Machine Learning Methods

4.1 PySpark MLlib

PySpark is a Python library by Scikit learn designed to process large datasets across multiple machines in a cluster, using distributed computing, which allows

scalability of computations from a single machine to a cluster of thousands of nodes, enabling faster data processing. PySpark's major strength is the ability to store data in memory (RAM) for fast access and computation, rather than relying on slow disk-based storage, which equally leads to faster data processing compared to other big data tools, like Hadoop MapReduce. PySpark also supports DataFrames, which are like tables in relational databases. They are distributed collections of data organized into named columns, making it easier to perform SQL-like operations on datasets, in this study, Pyspark is used to implement Linear Regression, Decision Tree Regressor, XGBoost Regressor and Random Forest Regressor.

4.2 Linear Regression

Linear Regression (LR) is the most common linear models used for prediction of continuous values. It models the relationship between the dependent variable (Y) and the independent variables (X) by fitting a linear equation to the data, to function effectively, LR models operate based on four major assumptions, namely, linearity, which assumes that the relationship between the independent and dependent variables is linear, this can be visually checked using scatter plots or by examining residual plots. The next assumption is Homoscedasticity, which assumes that the residuals have constant variance across all levels of the independent variable. The next assumption is that the relationship between the independent variables remain nondependent and uncorrelated, and lastly, is assumes a normal distribution among the variables. In this experiment, LR is used to access the linearity that relates the various features.

4.3 Generalized Linear Regression

GLR extends ordinary least squares (OLS) regression by allowing different response distributions and link functions. This makes GLR useful when house price data exhibit skewness, heteroscedasticity, or non-Gaussian distributions. Unlike standard Linear Regression which assumes, dependent variable (house price) is normally distributed, and that errors have constant variance (homoscedasticity), GLR relaxes these assumptions by modelling house prices using alternative probability distributions.

4.4 Random Forest Regressor

A Random Forest Regressor is a machine learning model used for predicting continuous values, like house prices. It works by constructing multiple decision trees during training and outputs the average prediction of the individual trees. This approach reduces overfitting and improves accuracy compared to using a single decision tree

4.5 XGBoost Regressor

XGBoost (Extreme Gradient Boosting) is a powerful, optimized gradient boosting algorithm designed for speed and performance (Chen & Guestrin, 2016). It is widely used in house price prediction due to its ability to handle non-linearity, high-dimensional data, and missing values efficiently.

4.6 Decision Tree Regressor

A Decision Tree Regressor is a non-parametric model that recursively splits the dataset into regions based on feature values. Each split is chosen to reduce the variance in the target variable (here, house prices), which means the model partitions the data into homogenous segments

4.7 Time Series Plot

A time series is a sequence of data points recorded over time at regular intervals. The experiment took a slight deviation to evaluate the suitability of the dataset for time series forecasting since it is commonly used in house price prediction and according to the time series plot in Figure 1, from the 1990s to early 2000s, house prices increased steadily but with some fluctuations, with a sharp increase around 2000 to 2008. After 2008, prices fluctuate around a high value and some major spikes can be seen. House price fluctuations continue till late 2020 where a bit of a decline can be seen in 2021.

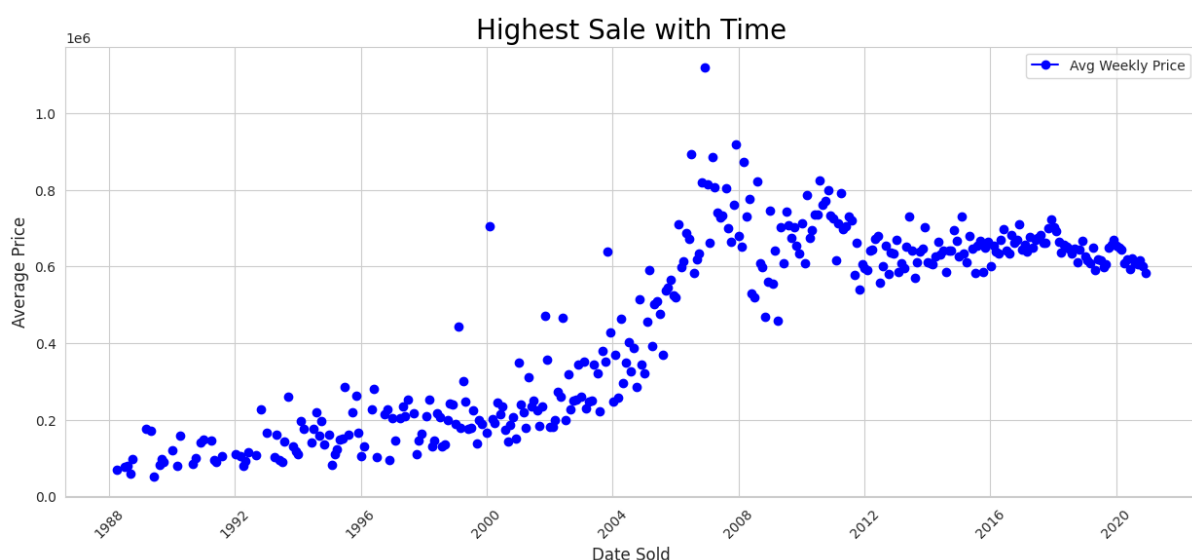


Figure 1. Showing rate of house sales with time

4.8 Tableau

Tableau is a powerful data visualization tool used for converting raw data into interactive and easy-to-understand visualizations. It helps organizations analyse their data and gain insights quickly, making it a popular choice for business intelligence (BI).

In this experiment, Tableau was used to create dashboards where the different insights on house prices were being analysed and visualized. Different visualizations such as Top 50 Suburbs by House Price, (Figure 2), were generated in this experiment.



Figure 2. Showing Top 50 expensive suburbs to live in Perth

5.0 Explanation of the Experiment

The experiment on the House Price Prediction begins by installing PySpark and FindSpark, then importation of other required default libraries and loading the dataset. Thereafter, an overview of the dataset is conducted, the schema of the dataset is inspected, with intent to uncover any errors or noise that could be found in the data. A Histogram plot of the raw data distribution using a Kernel Density Estimate (KDE), can be seen in Figure 3, notice the right-skewed distribution, indicating that the most available houses are inexpensive, at 500K, compared to the span of the whole data, house prices above 2million exists but are relatively few.

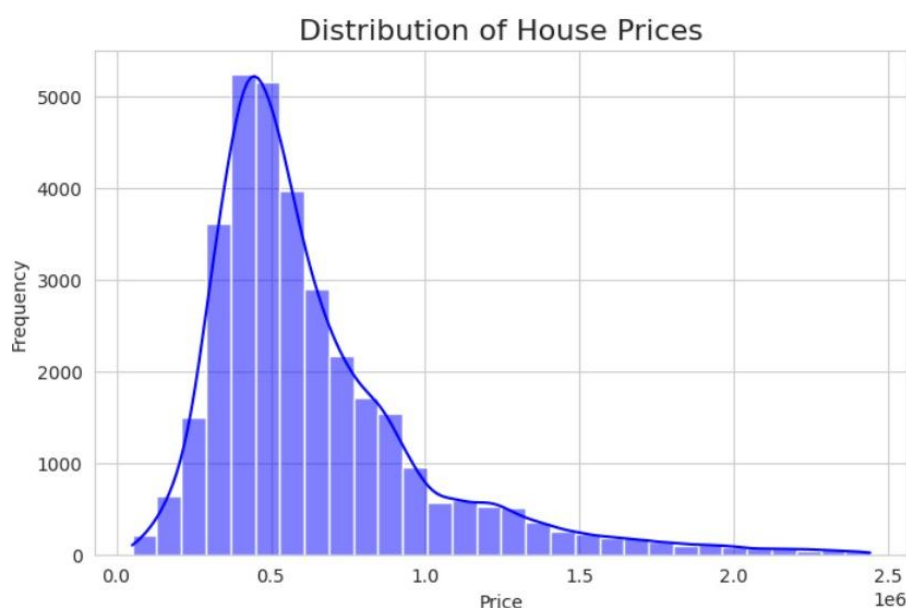


Figure 3. Showing the distribution of house prices in Perth

The analysis features five regression models namely, Linear Regression, Generalized Linear Regression, Random Forest, XGBoost and the Decision Tree Classifier all hosted in a model dictionary and tested at once. The evaluation is structured such that, after model prediction is made, different models can be selected for full evaluation with results and their appropriate visualizations, a default model is selected for the very first evaluation, and after the overall evaluation, a name of the best performing model is displayed.

5.1 Exploratory Data Analysis

Furthermore, to properly understand the data, a Correlation Matrix Heatmap (Figure 4), was used to explore the data, and from the findings, FLOOR_AREA (0.55) and BATHROOMS (0.38) have the strongest positive correlations with PRICE, meaning that larger floor areas and more bathrooms generally lead to higher prices. Also, CBD_DIST (-0.35) and NEAREST_SCH_RANK (-0.46) shows negative correlations, meaning that houses farther from the Central Business District and those near lower-ranked schools tend to be cheaper. Bedrooms and Bathrooms have a 0.56 correlation, which makes sense because, houses with more bedrooms usually have more bathrooms.

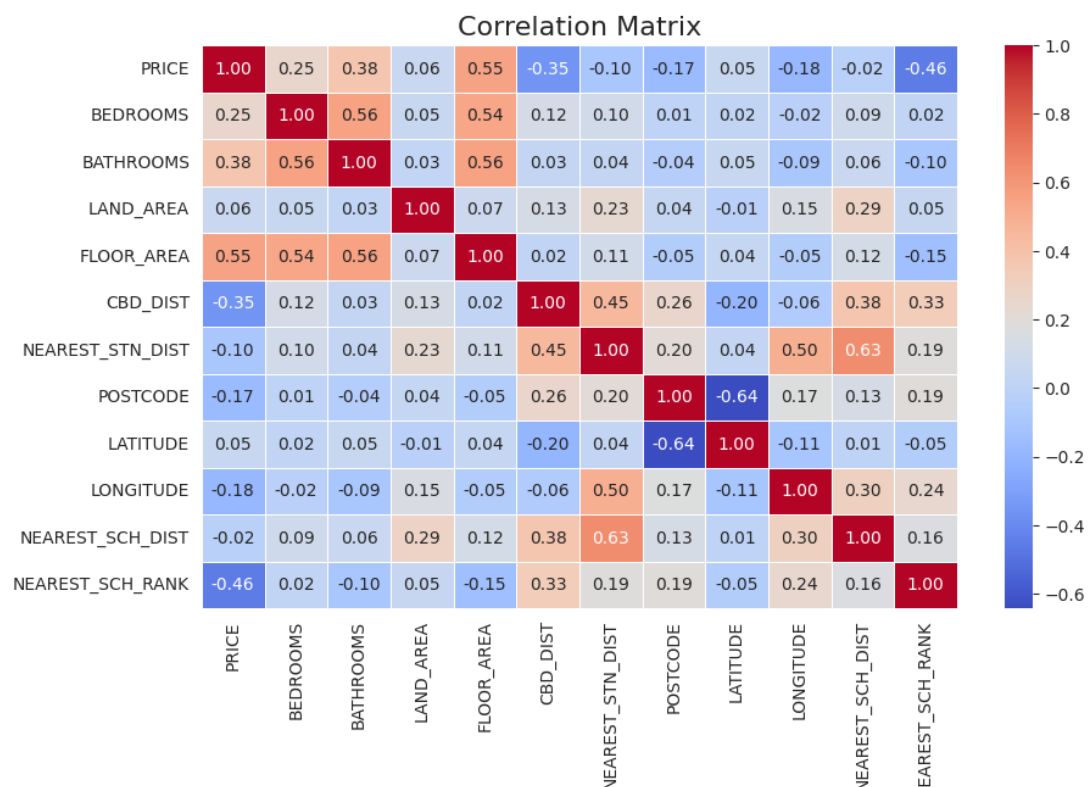


Figure 4. Showing a Correlational Heatmap of numerical values

Other Visualizations preformed included a Pairplot, which was used to preview a graphical correlation between the numerical data in the dataset, and a box plot visualization of all the numerical features was also performed with relation to PRICE, to spot outliers, a graphical representation of one of the boxplots of the BEDROOMS can be found in Figure 5, expanded and shown side by side, since outliers were suspected.

And as can be seen in the illustration above, House Prices generally arises as the number of bedrooms increase. Homes with more than 8 rooms have significantly higher prices. Houses with 4 or more bedrooms show wide Interquartile Ranges (IQRs), meaning that these categories have the most varying prices. Overall, high variation and outliers suggest that other factors like location, land area and other social facilities influence pricing beyond just bedroom count.

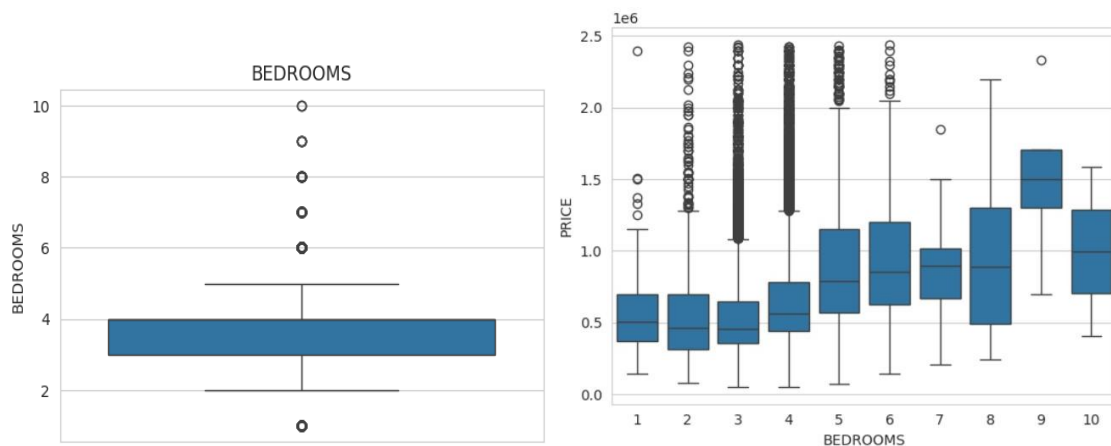


Figure 5. Showing box plots to observe outliers

5.2 Data Preprocessing

Data preprocessing is the process of preparing data to be fitted into the model. During data preparation it was discovered that there were 10,952 null values on the NEAREST_SCH_RANK, quite a number, which is 32.54% of the overall data. For this experiment, all the missing data was removed since the feature is not so critical and does not depend on other features, also the 32.54% mark stands on the high side.

5.3 Feature Engineering

This is the process of creating new features or transforming already existing ones to improve a model's performance. In the experiment, DATE_SOLD was transformed into YEAR_SOLD, and MONTH_SOLD, and BUILD_YEAR was cast from string to integer, and later transformed to generate the age of the building and stored as PROPERTY_AGE, this was later used to generate a category that tells if the house was old, moderate or new. The transformed features from YEAR_SOLD are used to create Moving Average (MA_) and Lag (LAG_), for a Time Series plot with trendlines representing the moving average trend of training and test data (Figure 6). The light blue region represents training data, the dark blue line indicates test data. The yellow section indicates the predicted future data while the red portion indicates the moving average trend for the test data.

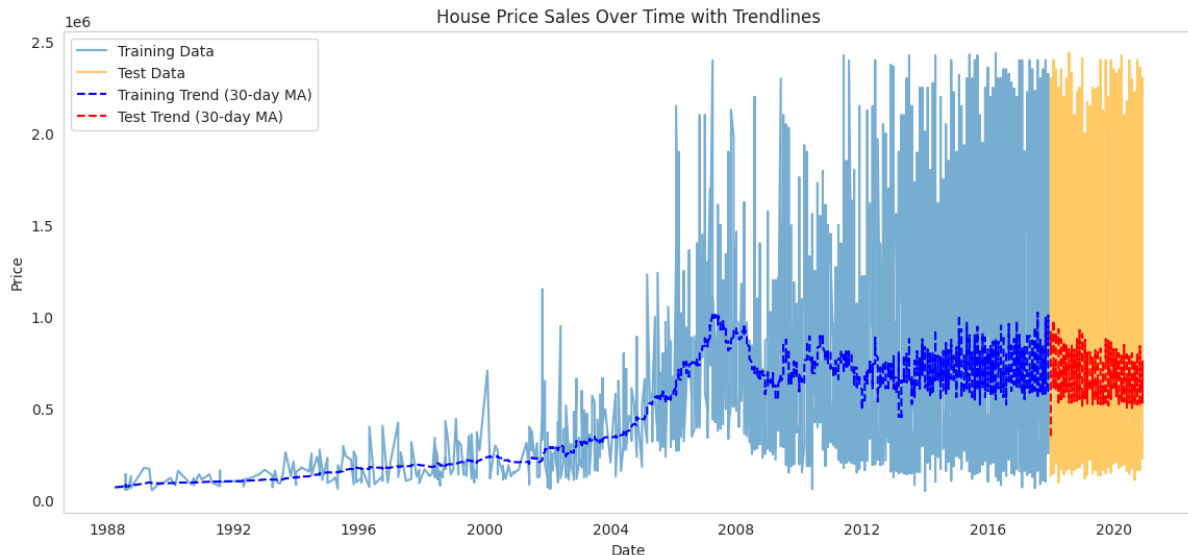


Figure 6. Showing a visualization of the Time Series Prediction

5.4 Data Encoding / Label Encoding

Data encoding refers to the process of converting categorical features into numerical values so that machine learning models can perform computations on them, the encoded labels are the string values. After creating PROPERTY_AGE, it is label encoded, and then one hot encoded into numerical values for prediction.

5.5 Model Preparation

At the model preparation phase, only numerical values are used, two copies of numerical variables excluding the target variable are saved with different names, one containing BUILD_AGE_INDEX, of datatype double, which is a produce of encoding the Old, Moderate, New elements, using StringIndexer, a feature from Apache Spark's MLlib (PySpark), into numbers, for the tree ensemble models like Random Forest, and the second copy contained AGE_VECTOR, of vector datatype, a product of applying the one hot encoder on BUILD_AGE_INDEX, to form a vector for the linear based models like Linear Regression.

6.0 Tableau Visualizations

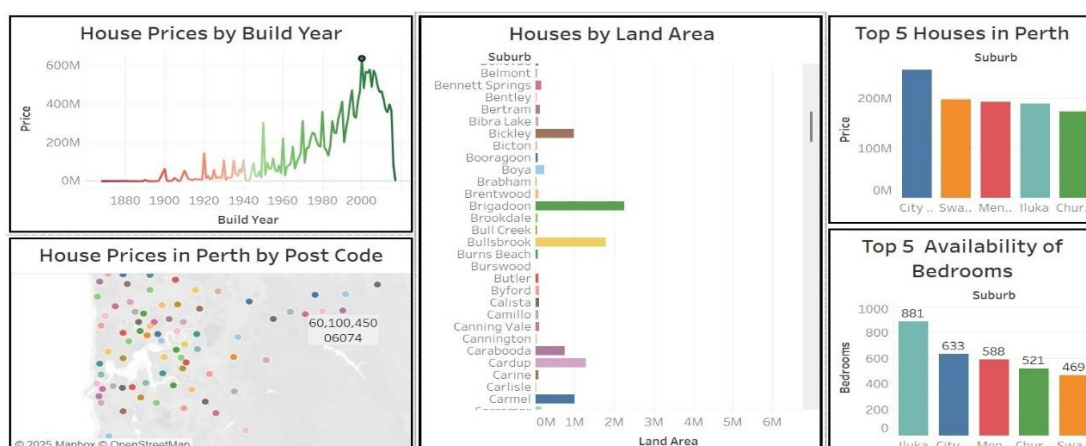


Figure 7. Showing a Tableau Dashboard

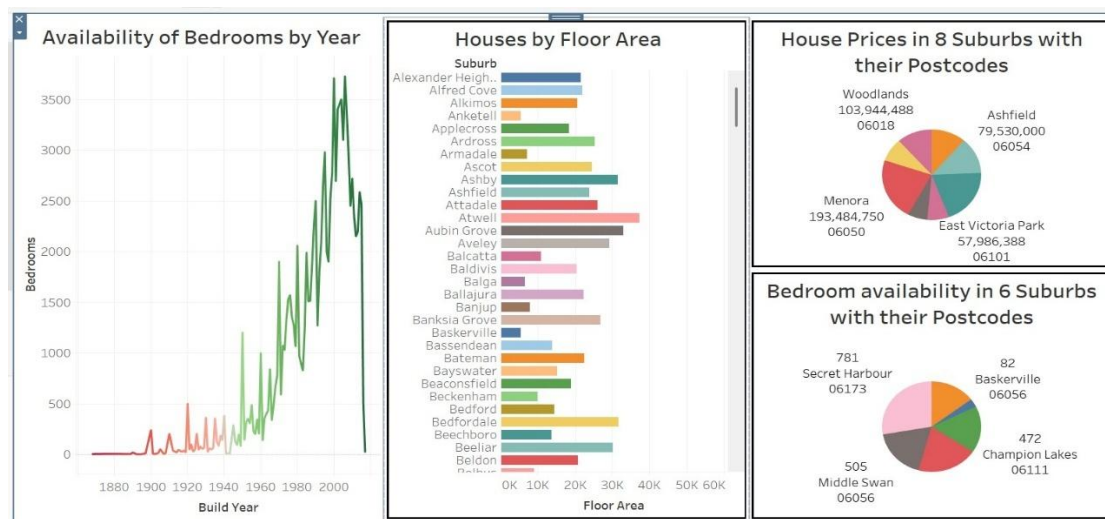


Figure 8. Showing another Tableau Dashboard visualization

7.0 Discussion of Results

After the analysis, the following outcome was observed, according to the models, based on their RMSE, R^2 Score and MAE.

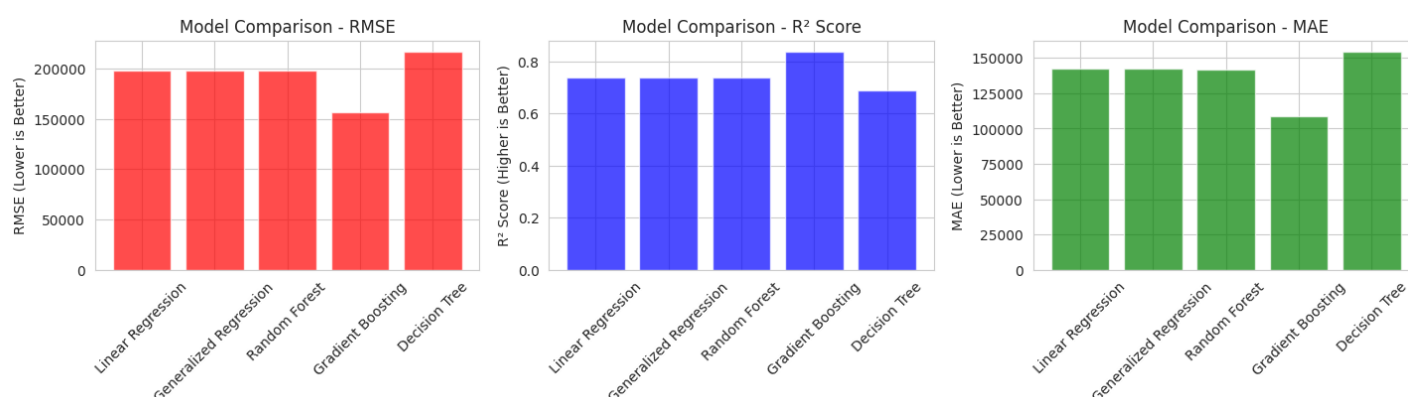


Figure 9. Showing model performance evaluation

7.1 RMSE

RMSE is a metric used to measure the error in a regression model, with lower values indicating better performance, and from the Histogram in Figure 9 Gradient Boosting performs best having the lowest RMSE. Linear Regression, Generalized Regression, and Random Forest have similar RMSE values, indicating that they perform similarly. Decision Tree has the highest RMSE, meaning it is the least accurate among the models.

7.2 R^2 Score

The R^2 score (coefficient of determination) measures how well a model explains the variance in the target variable, with values closer to 1 indicating better performance, and from the illustration in Figure 9, Gradient Boosting has the highest R^2

score, meaning it explains the most variance and is the best-performing model. Linear Regression, Generalized Regression, and Random Forest have similar R^2 scores, indicating they perform comparably. Decision Tree has the lowest R^2 Score, re-emphasizing its poor performance, as was seen in the RMSE chart.

7.3 MAE

MAE measures the average absolute difference between predicted and actual values, with lower values indicating better performance, and from the observation of results, Gradient Boosting has the lowest MAE, making it the most accurate model in terms of absolute error, re-emphasizing what was already seen in the R^2 Score.

7.4 Prediction Scatter Plot

A prediction scatter plot visually compares predicted values from a model to actual values. It is often used in regression analysis to fit a reference line (dashed red line), and from Figure 10, the data points form a roughly diagonal pattern, indicating linearity, meaning that the predicted house prices tend to follow the actual prices, suggesting that the model is generally performing well. From the image, 100% predictions are all the data points that fall directly under the reference line. Some points deviate significantly from the red line, indicating over- or under-predictions.



Figure 10. Showing a Scatter plot of the Predicted vs. Actual prices

7.5 Residuals Distribution

Residuals are the difference between the actual and predicted values in a model. The image in Figure 11 shows that the residuals are approximately symmetric and centred around zero, indicating a near-normal distribution. This indicates that the model does not have significant bias in predictions. Also, it can be observed that the skewness has been eliminated, which indicates that errors are distributed evenly across predictions. It can also be seen that there are few outliers in the overall data, and there are some residuals further from zero, particularly on the right. This could indicate a few over-predictions

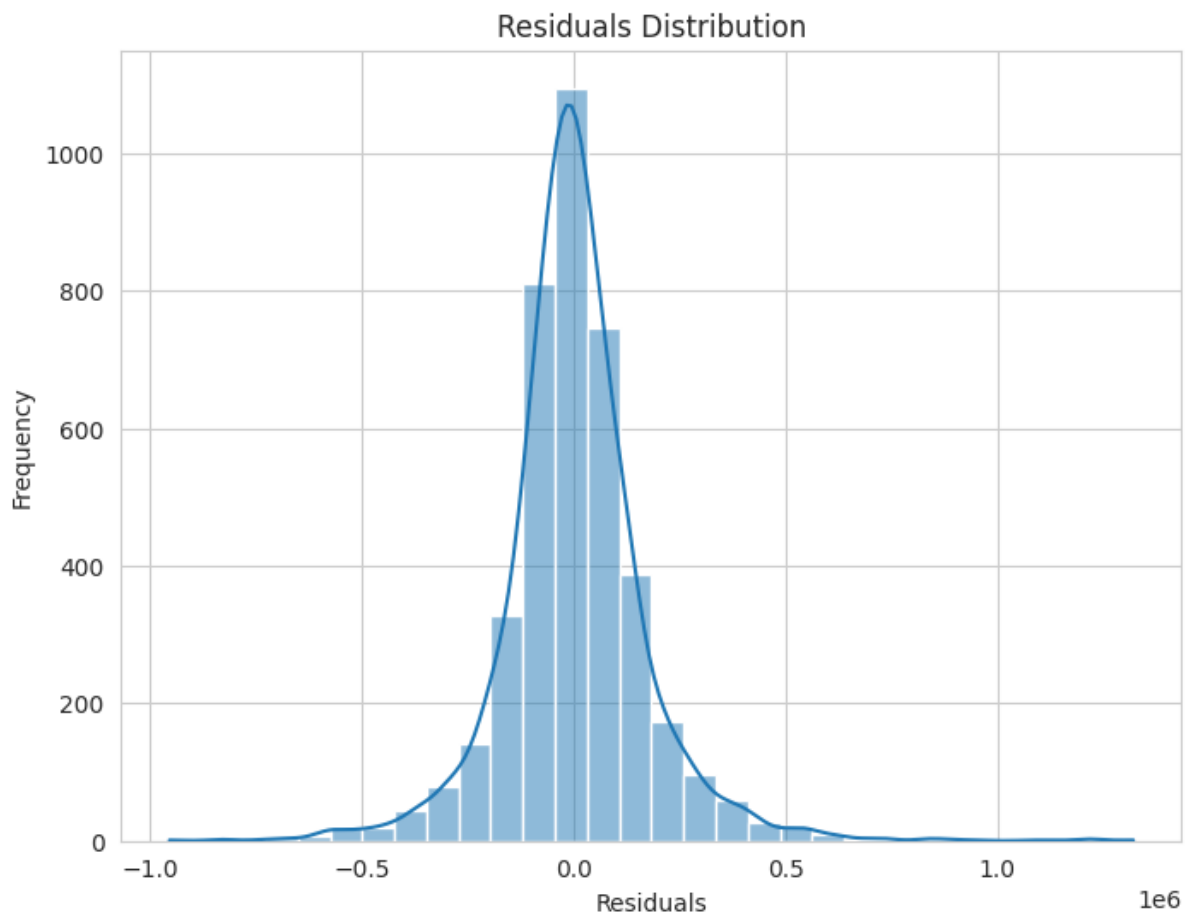


Figure 11. Showing a distribution of the residuals

7.6 Residual Scatter Plot

A residuals scatter plot shows the difference between actual and predicted house prices against predicted values. And from Figure 12, the residuals are fairly spread across all predicted price ranges, but variance increases for higher predicted prices, indicating Homoscedasticity, one of the core assumptions of Linear Regression models. Also, the wider spread of residuals at higher prices suggests the model may be less accurate for expensive houses.

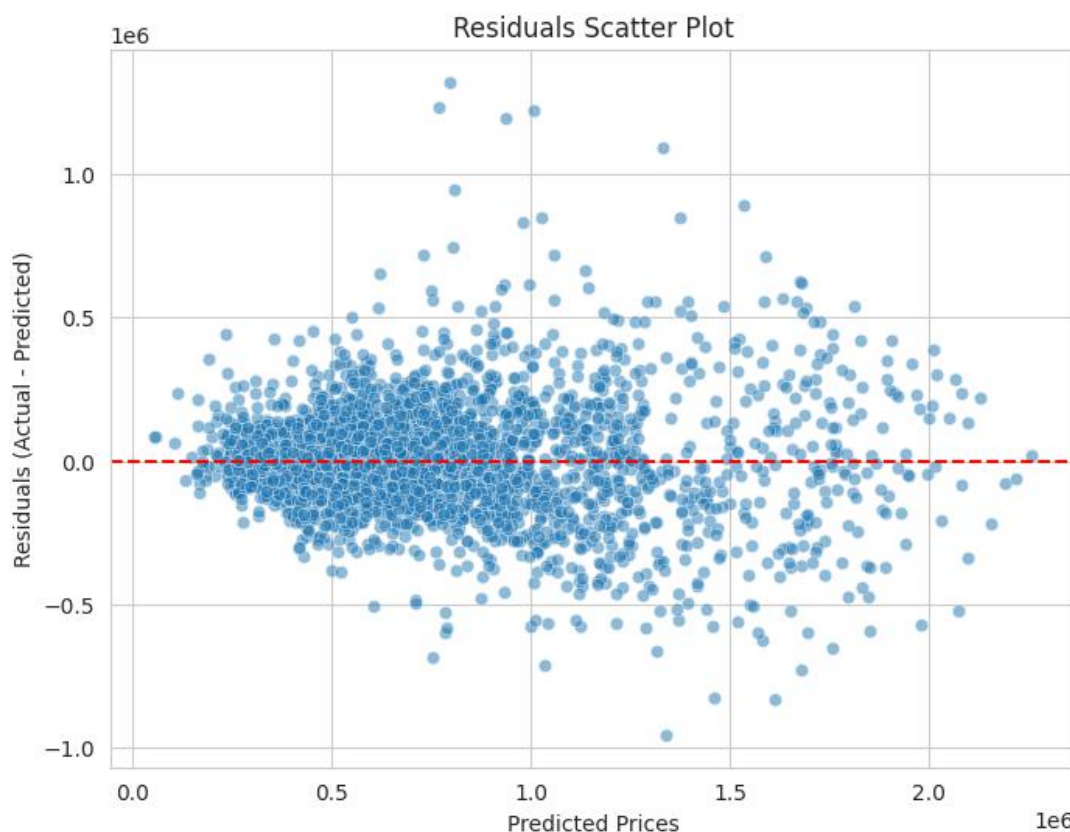


Figure 12. Showing a Scatter plot of the residuals

7.7 Ordinary Least Squares (OLS) Regression Results

OLS Regression is a fundamental statistical method used in linear regression to estimate the relationship between a dependent variable (target) and one or more independent variables (predictors). When R^2 is 1 the model is at the best performance, meaning that R^2 of 0.807 equates to a performance of 80.7% which means that the dependent variable (y) is well explained by the independent variables (X). From Figure 13 the total number of samples used in training is 16,601, number of independent variables was 23, and the degrees of freedom from after fitting the model was 16,577.

↩

OLS Regression Results

=====

Dep. Variable:

y

R-squared:

0.807

Model:

OLS

Adj. R-squared:

0.807

Method:

Least Squares

F-statistic:

3020.

Date:

Sun, 23 Mar 2025

Prob (F-statistic):

0.00

Time:

23:58:22

Log-Likelihood:

-2.2229e+05

No. Observations:

16601

AIC:

4.446e+05

Df Residuals:

16577

BIC:

4.448e+05

Df Model:

23

Covariance Type:

nonrobust

=====

coef

std err

t

P>|t|

[0.025

0.975]

const

-1.756e+05

3.21e+05

-0.548

0.584

-8.04e+05

4.53e+05

x1

-5775.7536

1609.833

-3.588

0.000

-8931.199

-2620.309

x2

3.855e+04

1676.383

22.996

0.000

3.53e+04

4.18e+04

x3

1.061e+04

1201.013

8.833

0.000

8254.881

1.3e+04

x4

1.37e+05

1720.421

79.609

0.000

1.34e+05

1.4e+05

Figure 13. Showing the Ordinary Least Squares result Page 16 | 49

8.0 Conclusion

This experiment uses Logistic Regression to predict house prices, disregarding the hedonistic pricing model and incorporating various input variables for model training. Applying log transformation to house prices should make the distribution more normal which equally impacts on the model predictions and improve the results. The best model explains 80.7% of the variance in y, making it a strong predictive model, however, further refinements like feature selection and transformations could further enhance performance. An overview of experiment results can be seen in Table 2 below. The learning curve visualisation in Figure 14 shows that model learning improves as the training data increases.

MODEL NAME	RMSE	R ²	MAE
Linear Regression	198144.594	0.740	142451.128
Generalized Regression	198144.594	0.740	142451.128
Random Forest	198049.056	0.741	141556.995
Gradient Boosting	156147.343	0.839	108728.197
Decision Tree	217043.615	0.688	154625.178

Table 2. Showing overall results of model performance

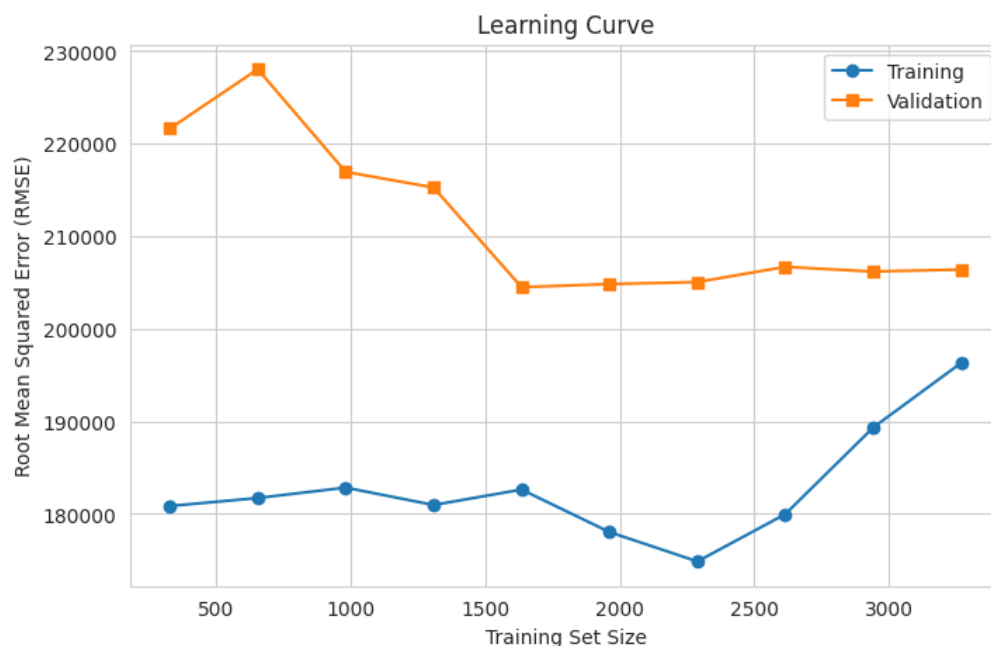


Figure 14. Showing the learning curve of the model

9.0 Further Research

In the time series plot, SARIMA models can be used to apply seasonality, and then predictions can be made using the time series models and the results compared with the traditional models. Another very remarkable improvement would be to implement the dataset on a Neural Network model to boost feature engineering and promote model performance.

10.0 Social / Ethical Impact

While predictive analytics in real estate offers significant benefits such as market transparency, economic growth, and better housing policies, it also raises ethical challenges like bias, privacy concerns, and market manipulation risks. Responsible AI practices, fair data usage policies, and open access to predictive insights are critical to ensuring socially beneficial and ethical implementation of this experiment.

11.0 References

- Dharmaraj, K. P., Gupta, P. K., Ajith, K., Kolli, M., Khare, S., & Panda, N. (2024). Real Estate Price Prediction Using PySpark MLlib. *2024 IEEE International Conference on Smart Power Control and Renewable Energy (ICSPCRE)*, 1–6.
<https://doi.org/10.1109/ICSPCRE62303.2024.10675142>
- Iqbal, R., Doctor, F., More, B., Mahmud, S., & Yousuf, U. (2020). Big data analytics: Computational intelligence techniques and application areas. *Technological Forecasting and Social Change*, 153, 119253.
<https://doi.org/10.1016/j.techfore.2018.03.024>
- Oladunni, T., & Sharma, S. (2016). Hedonic Housing Theory—A Machine Learning Investigation. *2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 522–527. <https://doi.org/10.1109/ICMLA.2016.0092>
- Sharma, H., Harsora, H., & Ogunleye, B. (2024). An Optimal House Price Prediction Algorithm: XGBoost. *Analytics*, 3(1), 30–45.
<https://doi.org/10.3390/analytics3010003>
- Ullah, F., & Sepasgozar, S. M. E. (2020). Key Factors Influencing Purchase or Rent Decisions in Smart Real Estate Investments: A System Dynamics Approach Using Online Forum Thread Data. *Sustainability*, 12(11), 4382.
<https://doi.org/10.3390/su12114382>

- Wei, C., Fu, M., Wang, L., Yang, H., Tang, F., & Xiong, Y. (2022). The Research Development of Hedonic Price Model-Based Real Estate Appraisal in the Era of Big Data. *Land*, 11(3), 334. <https://doi.org/10.3390/land11030334>
- Zhang, H., Li, Y., & Branco, P. (2024). Describe the house and I will tell you the price: House price prediction with textual description data. *Natural Language Engineering*, 30(4), 661–695. <https://doi.org/10.1017/S1351324923000360>
- Zhao, Y., Zhao, J., & Lam, E. Y. (2024). House Price Prediction: A Multi-Source Data Fusion Perspective. *Big Data Mining and Analytics*, 7(3), 603–620. <https://doi.org/10.26599/BDMA.2024.9020019>

Rosen, S. (1974). Hedonic prices and implicit markets: product differentiation in pure competition. *Journal of political economy*, 82(1), 34-55.

Appendix I

Dataset used for this experiment can be found at...

https://www.kaggle.com/datasets/syuzai/perth-house-prices?select=all_perth_310121.csv

GitHub link for the code used in the experiment at...

https://github.com/JaminUbuntu/IBOK_viz/blob/main/PySpark_Code_Benjamin.ipynb

Appendix II

1. Screenshots of PySpark / Hadoop Installations

```

jamin@jamin-VirtualBox:~$ mkdir temp
jamin@jamin-VirtualBox:~$ cd temp/
jamin@jamin-VirtualBox:~/temp$ pwd
/home/jamin/temp
jamin@jamin-VirtualBox:~/temp$ java -version
Command 'java' not found, but can be installed with:
sudo apt install openjdk-17-jre-headless # version 17.0.12+7-1ubuntu2~24.04, or
sudo apt install openjdk-21-jre-headless # version 21.0.4+7-1ubuntu2~24.04
sudo apt install default-jre # version 2:1.17-75
sudo apt install openjdk-11-jre-headless # version 11.0.24+8-1ubuntu3~24.04.1
sudo apt install openjdk-8-jre-headless # version 8u422-b05-1~24.04
sudo apt install openjdk-19-jre-headless # version 19.0.2+7-4
sudo apt install openjdk-20-jre-headless # version 20.0.2+9-1
sudo apt install openjdk-22-jre-headless # version 22~22ea-1
jamin@jamin-VirtualBox:~/temp$

```

- Checking the Java version in Linux

```

sudo apt install openjdk-11-jre-headless # version 11.0.24+8-1ubuntu3~24.04.1
sudo apt install openjdk-8-jre-headless # version 8u422-b05-1~24.04
sudo apt install openjdk-19-jre-headless # version 19.0.2+7-4
sudo apt install openjdk-20-jre-headless # version 20.0.2+9-1
sudo apt install openjdk-22-jre-headless # version 22~22ea-1
jamin@jamin-VirtualBox:~/temp$ sudo apt update
Warning: The unit file, source configuration file or drop-ins of apt-news.servi
e changed on disk. Run 'systemctl daemon-reload' to reload units.
Warning: The unit file, source configuration file or drop-ins of esm-cache.servi
ce changed on disk. Run 'systemctl daemon-reload' to reload units.
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://gb.archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://gb.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:4 http://gb.archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 126 kB in 1s (173 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
32 packages can be upgraded. Run 'apt list --upgradable' to see them.
jamin@jamin-VirtualBox:~/temp$

```

- Running the **\$ sudo apt update**

```

update-alternatives: using /usr/lib/jvm/java-8-openjdk-amd64/bin/xjc to provide
/usr/bin/xjc (xjc) in auto mode
Setting up openjdk-8-jdk:amd64 (8u422-b05-1~24.04) ...
update-alternatives: using /usr/lib/jvm/java-8-openjdk-amd64/bin/appletviewer to
provide /usr/bin/appletviewer (appletviewer) in auto mode
update-alternatives: using /usr/lib/jvm/java-8-openjdk-amd64/bin/jconsole to pro
vide /usr/bin/jconsole (jconsole) in auto mode
Processing triggers for libc-bin (2.39-0ubuntu8.3) ...
jamin@jamin-VirtualBox:~/temp$ sudo java -version
openjdk version "1.8.0_422"
OpenJDK Runtime Environment (build 1.8.0_422-8u422-b05-1~24.04-b05)
OpenJDK 64-Bit Server VM (build 25.422-b05, mixed mode)
jamin@jamin-VirtualBox:~/temp$

```

- Checking the java version using **\$ sudo java -version**

```
jamin@jamin-VirtualBox:~/temp$ pwd
/home/jamin/temp
jamin@jamin-VirtualBox:~/temp$ ls
hadoop-2.10.2-src  hadoop-2.10.2-src.tar.gz
jamin@jamin-VirtualBox:~/temp$ export JAVA_HOME=/usr/lib/jvm/default-java
jamin@jamin-VirtualBox:~/temp$ PATH=$JAVA_HOME/bin:$PATH
jamin@jamin-VirtualBox:~/temp$ export HADOOP_HOME=/home/jamin/temp/hadoop-2.10.2
-src
jamin@jamin-VirtualBox:~/temp$ PATH=$HADOOP_HOME/bin:$PATH
jamin@jamin-VirtualBox:~/temp$ hadoop
```

- Checking the Java path using **\$ PATH=\$JAVA_HOME/bin:\$PATH**

```
jamin@jamin-VirtualBox:~/Desktop/temp$ cd ..
jamin@jamin-VirtualBox:~/Desktop$ cd ..
jamin@jamin-VirtualBox:~$ export HADOOP_HOME=/home/jamin/Desktop
p/temp/hadoop-3.4.0
jamin@jamin-VirtualBox:~$ PATH=$HADOOP_HOME/bin:$PATH
jamin@jamin-VirtualBox:~$ hadoop
hadoop: command not found
jamin@jamin-VirtualBox:~$ PATH=$HADOOP_HOME/bin:$PATH
jamin@jamin-VirtualBox:~$ hadoop
Usage: hadoop [OPTIONS] SUBCOMMAND [SUBCOMMAND OPTIONS]
or      hadoop [OPTIONS] CLASSNAME [CLASSNAME OPTIONS]
where CLASSNAME is a user-provided Java class

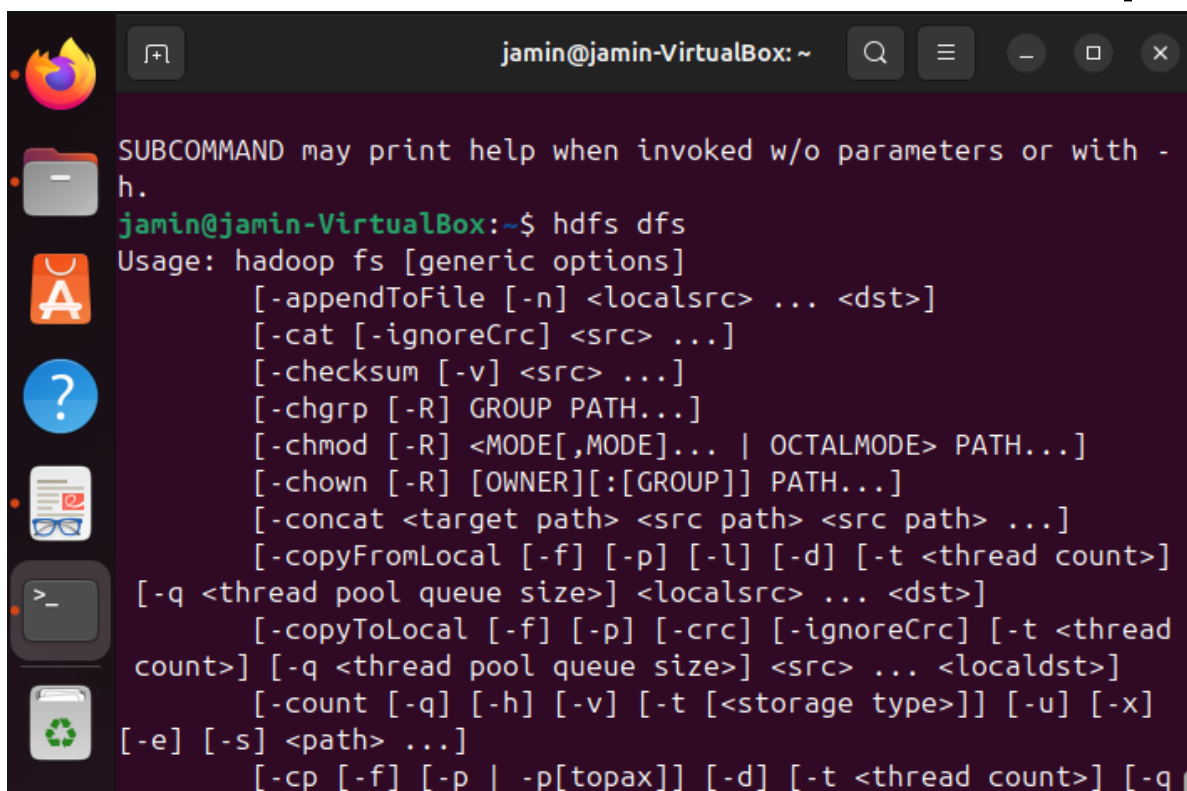
OPTIONS is none or any of:

buildpaths          attempt to add class files
                    from build tree
--config dir        Hadoop config directory
--debug            turn on shell script debug
                    mode
--help             usage information
hostnames list[,of,host,names] hosts to use in worker mode
```

- Installing Hadoop

```
jamin@jamin-VirtualBox:~/temp$ pwd
/home/jamin/temp
jamin@jamin-VirtualBox:~/temp$ echo Paris, FRance > cities.csv
jamin@jamin-VirtualBox:~/temp$ echo Berlin, Germany >> cities.csv
jamin@jamin-VirtualBox:~/temp$ echo London, England >> cities.csv
jamin@jamin-VirtualBox:~/temp$ echo Copenhagen, Denmark >> cities.csv
jamin@jamin-VirtualBox:~/temp$ echo Manchester, Englang >> cities.csv
jamin@jamin-VirtualBox:~/temp$ cat cities.csv
Paris, FRance
Berlin, Germany
London, England
Copenhagen, Denmark
Manchester, Englang
jamin@jamin-VirtualBox:~/temp$ hadoop
```

- Checking **\$ cat input/cities.csv**

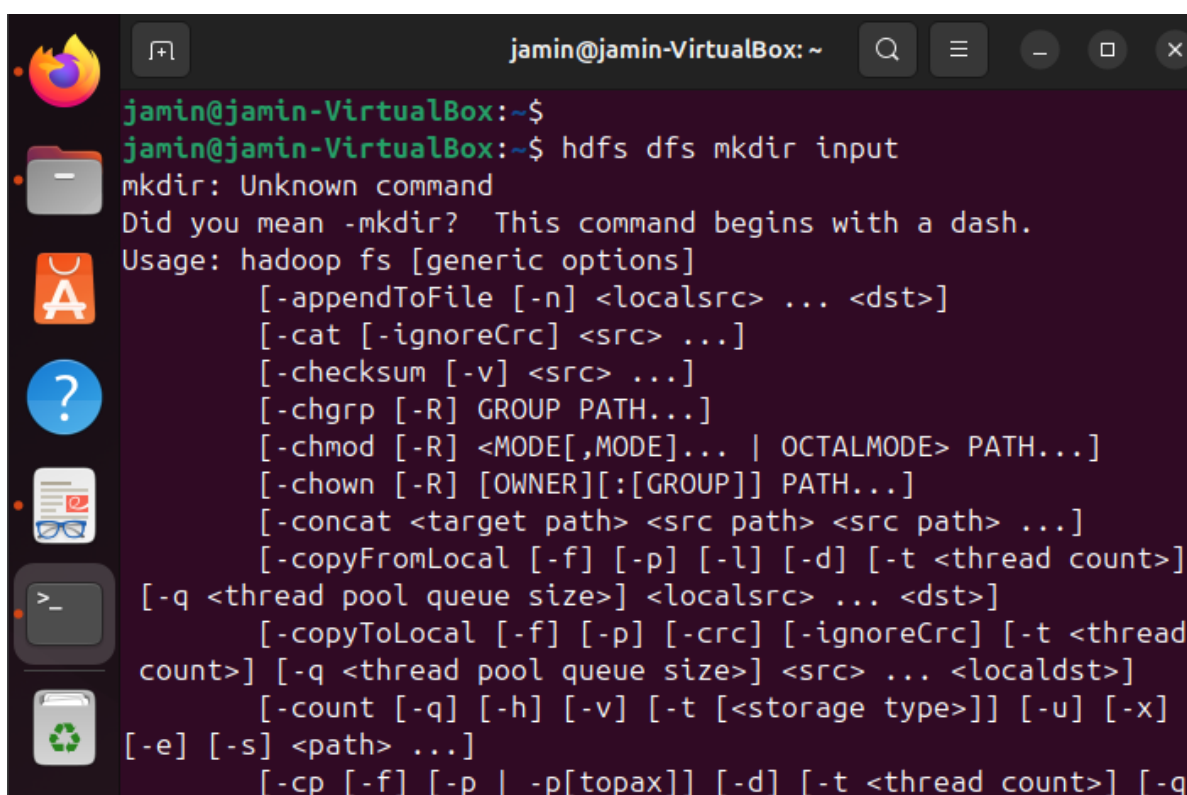


```

jamin@jamin-VirtualBox: ~
SUBCOMMAND may print help when invoked w/o parameters or with -h.
jamin@jamin-VirtualBox:~$ hdfs dfs
Usage: hadoop fs [generic options]
    [-appendToFile [-n] <localsrc> ... <dst>]
    [-cat [-ignoreCrc] <src> ...]
    [-checksum [-v] <src> ...]
    [-chgrp [-R] GROUP PATH...]
    [-chmod [-R] <MODE[,MODE]... | OCTALMODE> PATH...]
    [-chown [-R] [OWNER][:[GROUP]] PATH...]
    [-concat <target path> <src path> <src path> ...]
    [-copyFromLocal [-f] [-p] [-l] [-d] [-t <thread count>]
    [-q <thread pool queue size>] <localsrc> ... <dst>]
    [-copyToLocal [-f] [-p] [-crc] [-ignoreCrc] [-t <thread count>]
    [-q <thread pool queue size>] <src> ... <localdst>]
    [-count [-q] [-h] [-v] [-t [<storage type>]] [-u] [-x]
    [-e] [-s] <path> ...]
    [-cp [-f] [-p | -p[topax]] [-d] [-t <thread count>] [-q

```

- Running **HDFS**



```

jamin@jamin-VirtualBox:~$
jamin@jamin-VirtualBox:~$ hdfs dfs mkdir input
mkdir: Unknown command
Did you mean -mkdir? This command begins with a dash.
Usage: hadoop fs [generic options]
    [-appendToFile [-n] <localsrc> ... <dst>]
    [-cat [-ignoreCrc] <src> ...]
    [-checksum [-v] <src> ...]
    [-chgrp [-R] GROUP PATH...]
    [-chmod [-R] <MODE[,MODE]... | OCTALMODE> PATH...]
    [-chown [-R] [OWNER][:[GROUP]] PATH...]
    [-concat <target path> <src path> <src path> ...]
    [-copyFromLocal [-f] [-p] [-l] [-d] [-t <thread count>]
    [-q <thread pool queue size>] <localsrc> ... <dst>]
    [-copyToLocal [-f] [-p] [-crc] [-ignoreCrc] [-t <thread count>]
    [-q <thread pool queue size>] <src> ... <localdst>]
    [-count [-q] [-h] [-v] [-t [<storage type>]] [-u] [-x]
    [-e] [-s] <path> ...]
    [-cp [-f] [-p | -p[topax]] [-d] [-t <thread count>] [-q

```

- Running **HDFS DFS mkdir input**


```

jami@jami-VirtualBox: ~
registrydns run the registry DNS server

SUBCOMMAND may print help when invoked w/o parameters or with -h.

jami@jami-VirtualBox:~$ hadoop jar $HADOOP_HOME/share/hadoop/
mapreduce/hadoop-mapreduce-examples-3.4.0.jar wordcount input o
utput
2024-10-15 11:50:17,086 INFO impl.MetricsConfig: Loaded propert
ies from hadoop-metrics2.properties
2024-10-15 11:50:17,249 INFO impl.MetricsSystemImpl: Scheduled
Metric snapshot period at 10 second(s).
2024-10-15 11:50:17,251 INFO impl.MetricsSystemImpl: JobTracker
metrics system started
2024-10-15 11:50:17,495 INFO input.FileInputFormat: Total input
files to process : 1
2024-10-15 11:50:17,541 INFO mapreduce.JobSubmitter: number of
splits:1
2024-10-15 11:50:17,852 INFO mapreduce.JobSubmitter: Submitting
tokens for job: job_local1831592436_0001

```

- Running the **\$ hadoop jar \$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-2.7.3.jar wordcount input output**

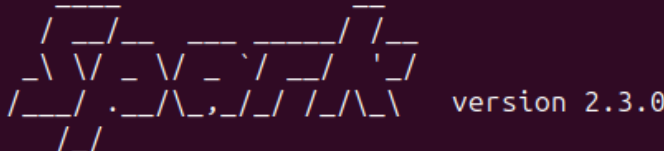
```

File Output Format Counters
  Bytes Written=108
jami@jami-VirtualBox:~$ hdfs dfs -ls output
Found 2 items
-rw-r--r--  1 jami jami          0 2024-10-15 11:50 output/_
SUCCESS
-rw-r--r--  1 jami jami       96 2024-10-15 11:50 output/p
art-r-00000
jami@jami-VirtualBox:~$ hdfs dfs -getmerge output /tmp/result
jami@jami-VirtualBox:~$ cat /tmp/result
Berlin, 1
Cophenegen,      1
Denmark 1
England 2
France  1
Germany 1
London, 1
Manchester,      1
Paris,  1
jami@jami-VirtualBox:~$

```

- Listing Categorizations


```
jamin@jamin-VirtualBox: ~  
/temp/spark-2.3.0-bin-hadoop2.7  
jamin@jamin-VirtualBox:~$ PATH=$SPARK_HOME/bin:$PATH  
jamin@jamin-VirtualBox:~$ spark-shell  
2024-10-15 15:45:15 WARN Utils:66 - Your hostname, jamin-Virtu  
alBox resolves to a loopback address: 127.0.1.1; using 10.0.2.1  
5 instead (on interface enp0s3)  
2024-10-15 15:45:15 WARN Utils:66 - Set SPARK_LOCAL_IP if you  
need to bind to another address  
2024-10-15 15:45:16 WARN NativeCodeLoader:62 - Unable to load  
native-hadoop library for your platform... using builtin-java c  
lasses where applicable  
Setting default log level to "WARN".  
To adjust logging level use sc.setLogLevel(newLevel). For Spark  
R, use setLogLevel(newLevel).  
Spark context Web UI available at http://10.0.2.15:4040  
Spark context available as 'sc' (master = local[*], app id = lo  
cal-1729003522206).  
Spark session available as 'spark'.  
Welcome to
```



```
version 2.3.0
```

```
Using Scala version 2.11.8 (OpenJDK 64-Bit Server VM, Java 1.8.  
0_422)  
Type in expressions to have them evaluated.  
Type :help for more information.
```

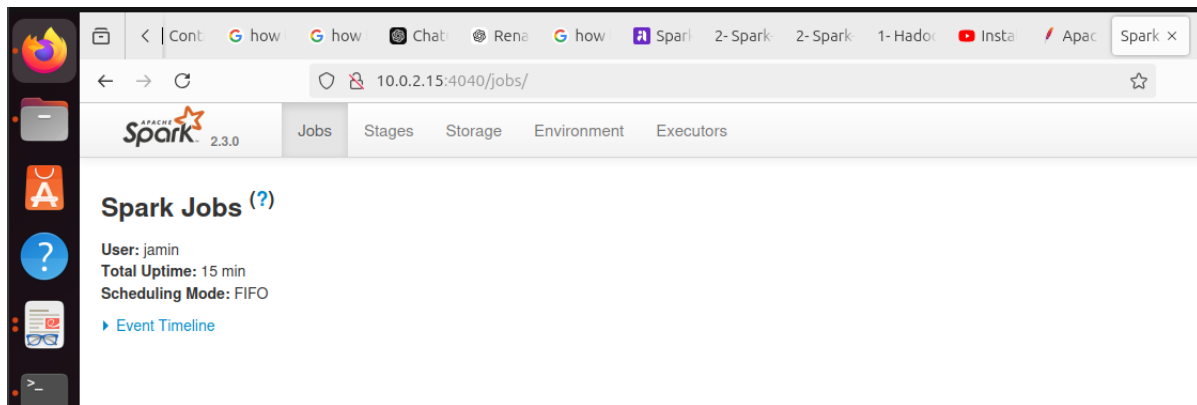
```
scala>
```

- Installation of PySpark

```
scala> sc.textFile("input").flatMap(line => line.split(" ")).map(word => (word, 1)).reduceByKey(_ + _).collect().foreach(println)
(Berlin,,1)
(Manchester,,1)
(London,,1)
(Denmark,1)
(Cophenegen,,1)
(France,1)
(Germany,1)
(Paris,,1)
(England,2)

scala>
```

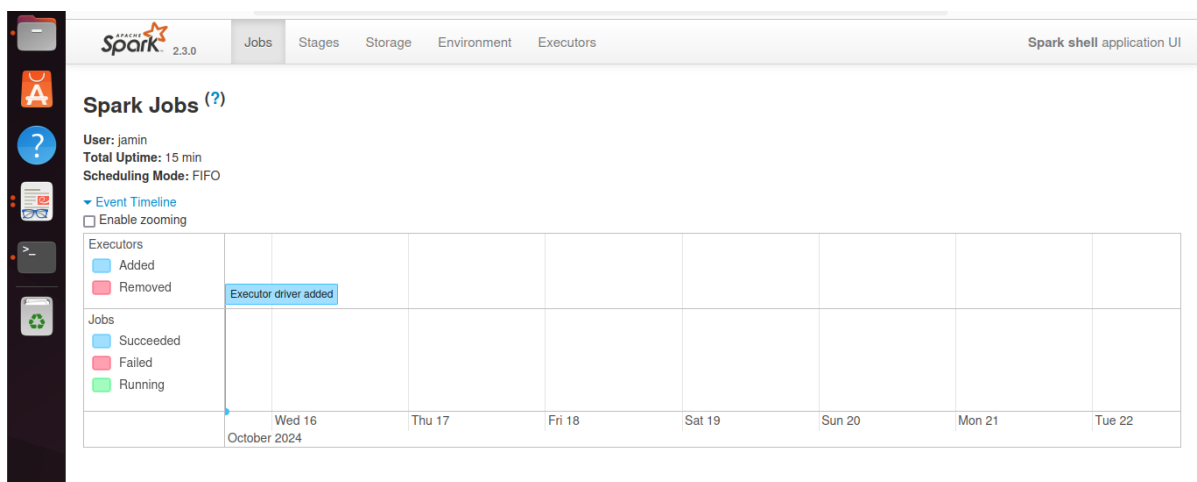
- Showing **Mapreduce**



Spark Jobs (?)

User: jamin
Total Uptime: 15 min
Scheduling Mode: FIFO

[Event Timeline](#)



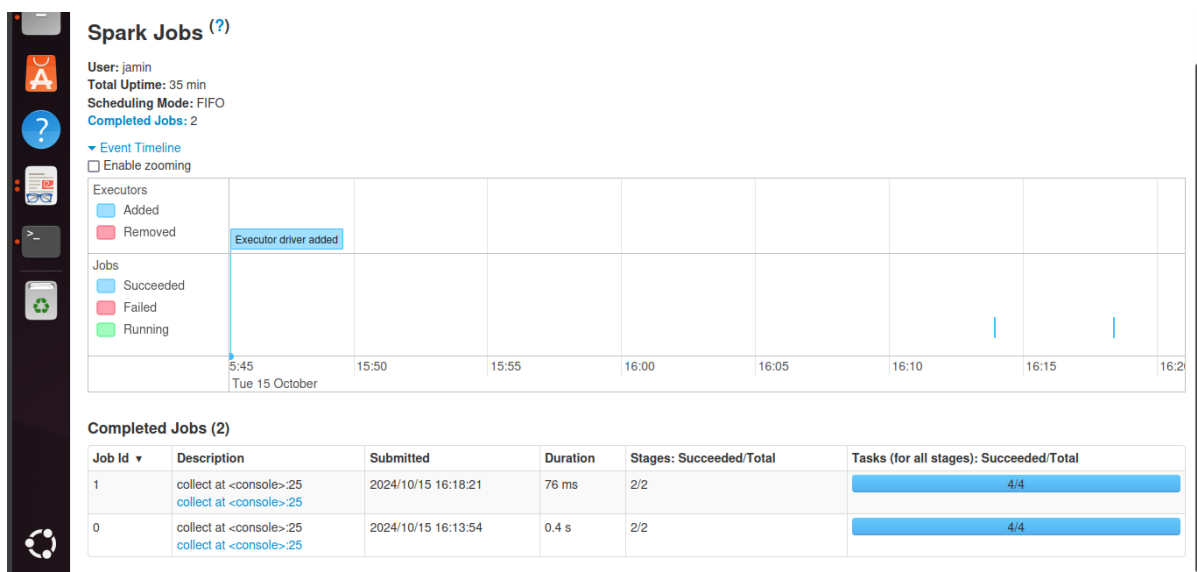
Spark Jobs (?)

User: jamin
Total Uptime: 15 min
Scheduling Mode: FIFO

[Event Timeline](#)

☐ Enable zooming

Executors	Jobs	Timeline
Added	Succeeded	Wed 16 October 2024
Removed	Failed	Thu 17
	Running	Fri 18
		Sat 19
		Sun 20
		Mon 21
		Tue 22



Spark Jobs (?)

User: jamin
Total Uptime: 35 min
Scheduling Mode: FIFO
Completed Jobs: 2

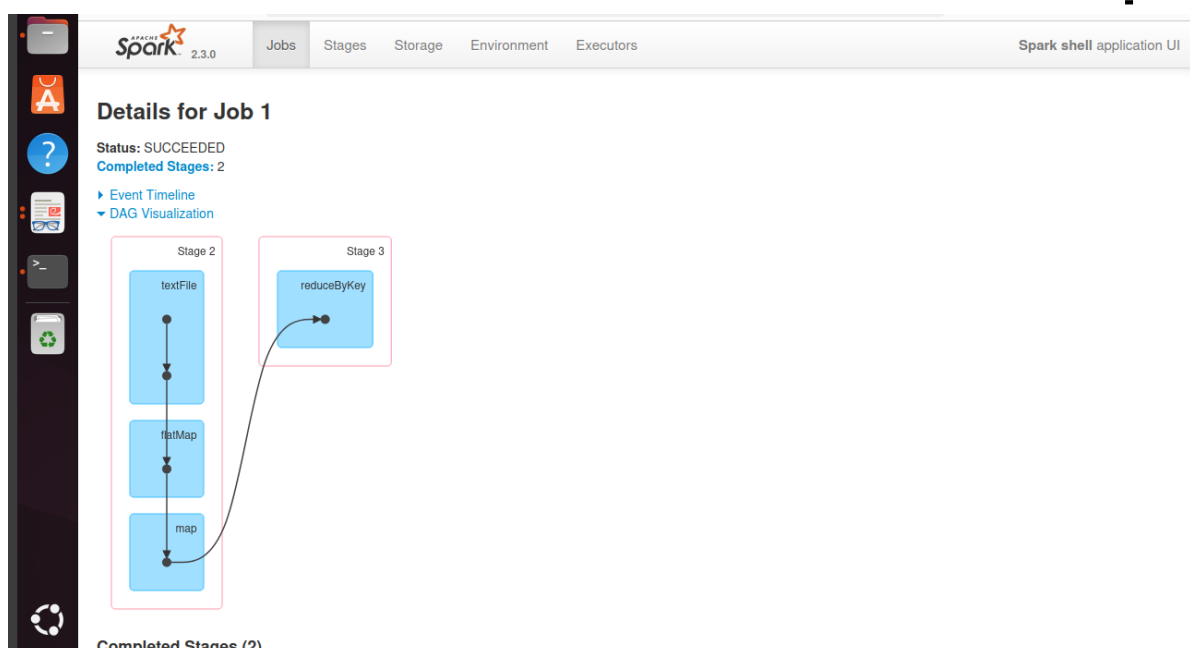
[Event Timeline](#)

☐ Enable zooming

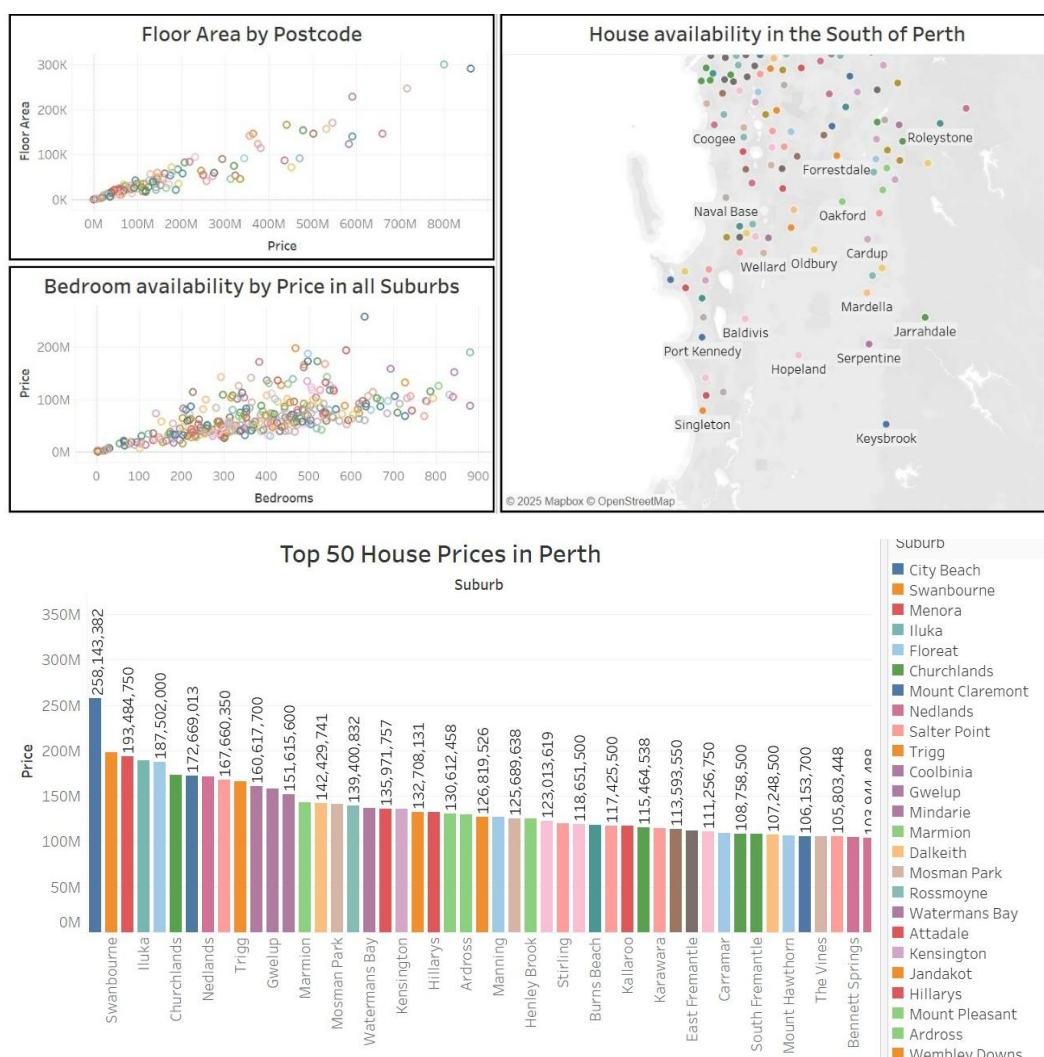
Executors	Jobs	Timeline
Added	Succeeded	5:45 Tue 15 October
Removed	Failed	15:50
	Running	15:55
		16:00
		16:05
		16:10
		16:15
		16:20

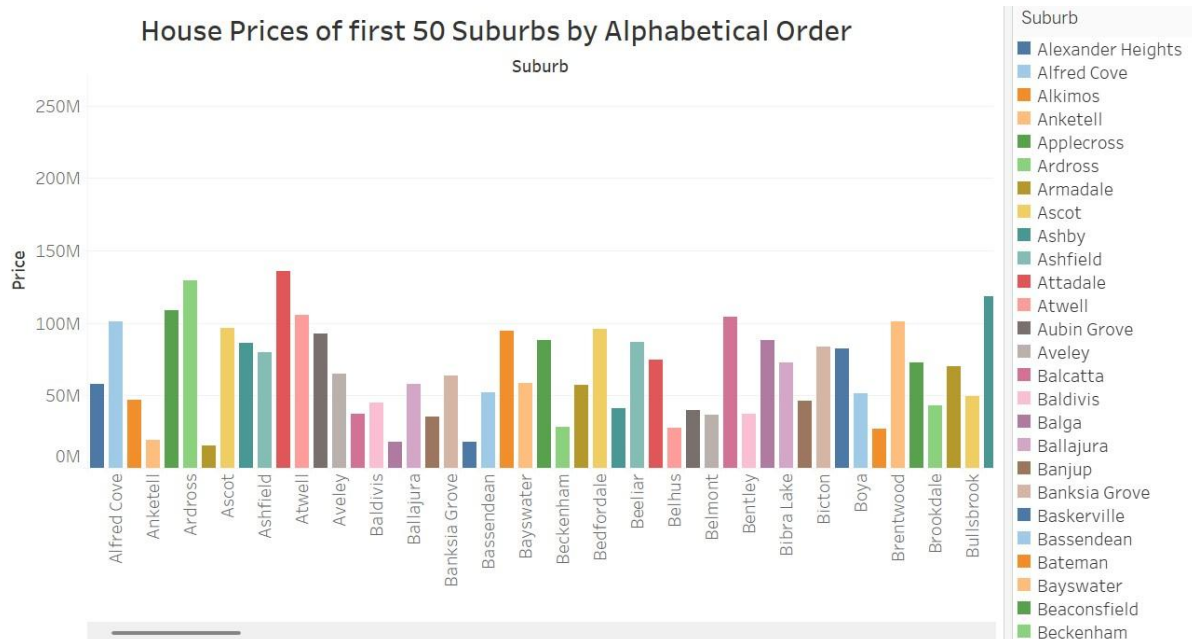
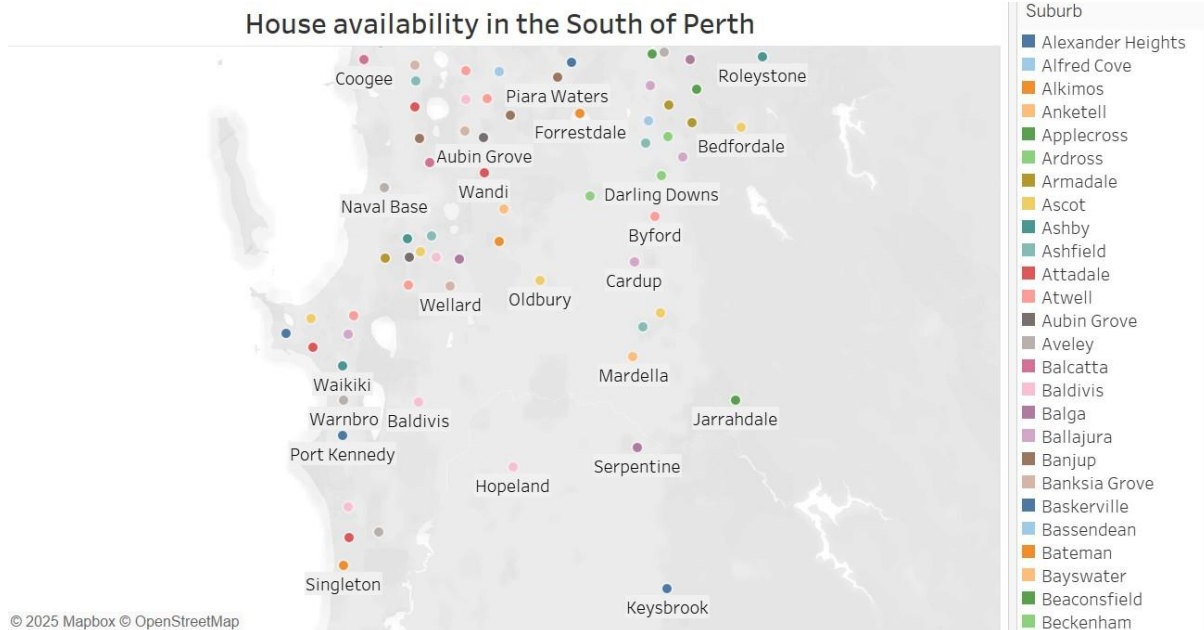
Completed Jobs (2)

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
1	collect at <console>:25 collect at <console>:25	2024/10/15 16:18:21	76 ms	2/2	4/4
0	collect at <console>:25 collect at <console>:25	2024/10/15 16:13:54	0.4 s	2/2	4/4



Screenshots of Tableau Visualisations





Screenshots of Python Codes

```
Collecting sktime
  Downloading sktime-0.36.0-py3-none-any.whl.metadata (34 kB)
Requirement already satisfied: joblib<1.5,>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from sktime) (1.4.2)
Requirement already satisfied: numpy<2.3,>=1.21 in /usr/local/lib/python3.11/dist-packages (from sktime) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from sktime) (24.2)
Requirement already satisfied: pandas<2.3.0,>=1.1 in /usr/local/lib/python3.11/dist-packages (from sktime) (2.2.2)
Collecting scikit-base<0.13.0,>=0.6.1 (from sktime)
  Downloading scikit-base-0.12.0-py3-none-any.whl.metadata (8.5 kB)
Requirement already satisfied: scikit-learn<1.7.0,>=0.24 in /usr/local/lib/python3.11/dist-packages (from sktime) (1.6.1)
Requirement already satisfied: scipy<2.0.0,>=1.2 in /usr/local/lib/python3.11/dist-packages (from sktime) (1.14.1)
Requirement already satisfied: python-dateutil<2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas<2.3.0,>=1.1->sktime) (2.8.2)
Requirement already satisfied: pytz<=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas<2.3.0,>=1.1->sktime) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas<2.3.0,>=1.1->sktime) (2025.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn<1.7.0,>=0.24->sktime) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil<2.8.2->pandas<2.3.0,>=1.1->sktime) (1.17.0)
Downloading sktime-0.36.0-py3-none-any.whl (36.9 MB)
36.9/36.9 MB 17.8 MB/s eta 0:00:00
Downloading scikit-base-0.12.0-py3-none-any.whl (141 kB)
141.5/141.5 kB 8.8 MB/s eta 0:00:00
Installing collected packages: scikit-base, sktime
Successfully installed scikit-base-0.12.0 sktime-0.36.0
```

```
PySpark Code Benjamin.ipynb
```

File Edit View Insert Runtime Tools Help

Commands + Code + Text

RAM Disk

```
[3] !pip install findspark
```

Collecting findspark
Downloading findspark-2.0.1-py2.py3-none-any.whl.metadata (352 bytes)
Downloading findspark-2.0.1-py2.py3-none-any.whl (4.4 kB)
Installing collected packages: findspark
Successfully installed findspark-2.0.1

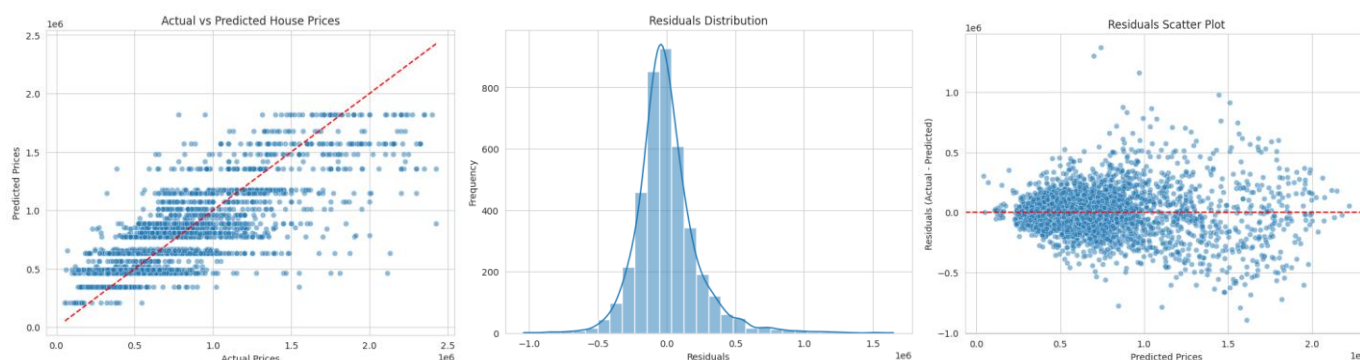
```
[4] import findspark
findspark.init()
findspark.find()
```

'/usr/local/lib/python3.11/dist-packages/pyspark'

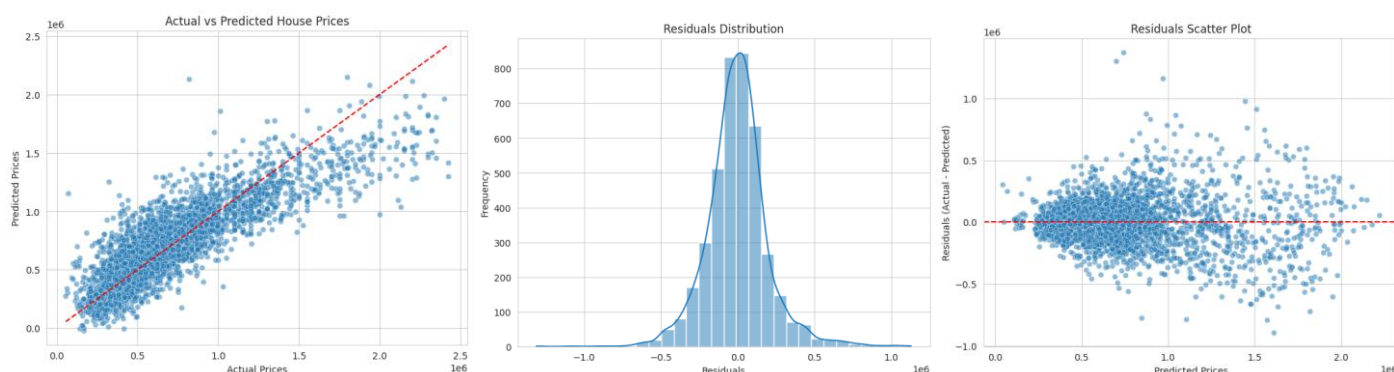
```
[5] from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('SparkByExamples.com').getOrCreate()
```

```
[7] df = spark.read.csv('/content/all_perth_310121.csv', header=True, inferSchema=True)
df.show()
```

EST_STN	NEAREST_STN_DIST	DATE_SOLD	POSTCODE	LATITUDE	LONGITUDE	NEAREST_SCH	NEAREST_SCH_DIST	NEAREST_SCH_RANK
ral ...	1800	09-2018	NULL	NULL	NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
Station	4900	02-2019	NULL	NULL	NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
Station	1900	06-2015	NULL	NULL	NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL



Showing plots for the Decision Tree model



Showing plots for Linear Regression model

Appendix III

The pasted Code...

```
!pip install pyspark
!pip install findspark
import findspark
findspark.init()
findspark.find()
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("HousePricePred").getOrCreate()
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.regression import LinearRegression
from pyspark.ml.regression import RandomForestRegressor
from pyspark.ml.regression import GBRegressor
from pyspark.ml.regression import GeneralizedLinearRegression
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
import matplotlib.pyplot as plt
import seaborn as sns

# Load Data
df = spark.read.csv("/content/all_perth_310121.csv", header=True,
inferSchema=True, multiLine=True, escape="\")
df.show()
df.printSchema()
df.describe().show()
from pyspark.sql.functions import count, when, col, isnan

df.select([
    count(when(col(c).isNull() | isnan(col(c))), c)).alias(c)
    for c in df.columns
]).show()

df_cleaned = df.dropna()
```

```
df_cleaned.select([
    count(when(col(c).isNull() | isnan(col(c))), c)).alias(c)
    for c in df_cleaned.columns
]).show()

df_cleaned.show()

# Select only numeric columns for Visualuzation
numeric_columns = [col_name for col_name, dtype in df.dtypes if dtype
in ('int', 'double', 'float', 'bigint')]

# Assign numeric columns to a new DataFrame
df_numeric = df.select(numeric_columns)

# Show result
df_numeric.columns

sns.set_style('whitegrid') # For grids on numerical plots

# Convert to Pandas
df_pd = df_numeric.select("*").toPandas()

# Compute correlation matrix
corr_matrix = df_pd.corr()

# Plot Heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm",
linewidths=0.5)
plt.title("Correlation Matrix", fontsize=16)
plt.show()

# Sample a subset of data
df_sample = df_pd.sample(frac=0.1, random_state=42)
```



```
# Pairplot
sns.pairplot(df_sample, diag_kind="kde")
plt.suptitle("Pairplot of Features", y=1.02, fontsize=16)
plt.show()

import seaborn as sns
import matplotlib.pyplot as plt

# Select only numeric columns
numeric_cols = [col for col, dtype in df.dtypes if dtype in ('int',
'double')]

# Convert to Pandas for visualization
df_numeric = df.select(numeric_cols).toPandas()

# Plot each numeric column in a separate row
for col in numeric_cols:
    plt.figure(figsize=(6, 4)) # Set individual figure size
    sns.boxplot(y=df_numeric[col])
    plt.title(col) # Set title for each plot
    plt.xticks([]) # Remove x-axis labels for clarity
    plt.show() # Show each plot separately

sns.boxplot(x=df_pd['BEDROOMS'], y=df_pd['PRICE'])
plt.show()

sns.boxplot(x=df_pd['BATHROOMS'], y=df_pd['PRICE'])
plt.show()

plt.figure(figsize=(8, 5))
sns.histplot(df_pd["PRICE"], bins=30, kde=True, color="blue")
plt.title("Distribution of House Prices", fontsize=16)
```

```
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()

df_ready = df_cleaned.drop("ADDRESS", "SUBURB", "NEAREST_STN",
, "GARAGE", "POSTCODE", "NEAREST_SCH")
df_ready.show()

from pyspark.sql.functions import regexp_replace

# Remove the \r from the DATE_SOLD column
df_ready = df_ready.withColumn("DATE_SOLD",
regexp_replace("DATE_SOLD", "\\r", ""))

# Display the result
df_ready.show()

from pyspark.sql.functions import to_date, year, month, dayofmonth,
unix_timestamp

# Convert DATE_SOLD to proper date format
df_ready = df_ready.withColumn("DATE_SOLD", to_date("DATE_SOLD", "MM-
yyyy"))

# Extract time components
df_ready = df_ready.withColumn("YEAR_SOLD", year("DATE_SOLD")) \
    .withColumn("MONTH_SOLD", month("DATE_SOLD"))

# Create a timestamp
df_t = df_ready.withColumn("DATE_TIMESTAMP",
unix_timestamp("DATE_SOLD"))

# Display the result
df_t.select("DATE_SOLD", "YEAR_SOLD", "MONTH_SOLD",
"DATE_TIMESTAMP").show()
```

```
df_t.show()
```

```
df_t.printSchema()
```

```
from pyspark.sql.functions import col
```

```
# Convert to integer
```

```
df_t = df_t.withColumn("BUILD_YEAR", col("BUILD_YEAR").cast("int"))
```

```
df_t.printSchema()
```

```
from pyspark.sql.functions import col
```

```
# Convert to integer
```

```
df_t = df_t.withColumn("BUILD_YEAR", col("BUILD_YEAR").cast("int"))
```

```
df_t.printSchema()
```

```
from pyspark.sql.functions import when
```

```
# Replace invalid values with null and cast as integer
```

```
df_t = df_t.withColumn("BUILD_YEAR",  
                        when(col("BUILD_YEAR").rlike("^[0-9]{4}$"),  
col("BUILD_YEAR").cast("int"))  
                        .otherwise(None))
```

```
df_t.show()
```

```
from pyspark.sql.functions import year, lit
```

```
# Calculate property age
```

```
df_t = df_t.withColumn("PROPERTY_AGE", year(lit("2025-01-01")) -  
col("BUILD_YEAR"))
```

```
df_t.show()
```

```
df_t.printSchema()
```

```
from pyspark.sql.functions import when
# Create age categories
df_t = df_t.withColumn("AGE_CATEGORY",
                        when(col("PROPERTY_AGE") <= 10, "New")
                        .when((col("PROPERTY_AGE") > 10) &
                             (col("PROPERTY_AGE") <= 30), "Moderate")
                        .otherwise("Old"))
```

```
df_t.show()
```

```
df_t.printSchema()
```

```
# Sort by date to ensure chronological order
df_t2 = df_t.orderBy("DATE_SOLD")
df_t2.show()
```

```
from pyspark.sql.window import Window
from pyspark.sql.functions import lag
```

```
# Define a window ordered by date
window_spec = Window.orderBy("DATE_SOLD")
```

```
# Add lag features like previous month and quarter price
df_t2 = df_t2.withColumn("LAG_1", lag("PRICE", 1).over(window_spec))
# 1 time period behind
df_t2 = df_t2.withColumn("LAG_3", lag("PRICE", 3).over(window_spec))
# 3 time periods behind
```

```
from pyspark.sql.functions import avg
# 3-period moving average
```

```
df_t2 = df_t2.withColumn("MA_3",
    avg("PRICE").over(window_spec.rowsBetween(-2, 0)))

# 6-period moving average
df_t2 = df_t2.withColumn("MA_6",
    avg("PRICE").over(window_spec.rowsBetween(-5, 0)))

from pyspark.sql.functions import quarter, weekofyear, dayofweek
# Add seasonality features
df_t2 = df_t2.withColumn("QUARTER", quarter("DATE_SOLD"))
df_t2 = df_t2.withColumn("WEEK_OF_YEAR", weekofyear("DATE_SOLD"))
df_t2 = df_t2.withColumn("DAY_OF_WEEK", dayofweek("DATE_SOLD"))

# Split chronologically
train_df = df_t2.filter(df_t2.YEAR < 2018)
test_df = df_t2.filter(df_t2.YEAR >= 2018)

# Convert to Pandas
train_pd = train_df.select("DATE_SOLD", "PRICE", "MA_3", "MA_6",
    "LAG_1", "LAG_3", "QUARTER", "WEEK_OF_YEAR").toPandas()
test_pd = test_df.select("DATE_SOLD", "PRICE", "MA_3", "MA_6",
    "LAG_1", "LAG_3", "QUARTER", "WEEK_OF_YEAR").toPandas()

# Set date as index for time series analysis
train_pd.set_index('DATE_SOLD', inplace=True)
test_pd.set_index('DATE_SOLD', inplace=True)

import matplotlib.pyplot as plt

# Example visualization
plt.figure(figsize=(14, 6))
plt.plot(train_pd['PRICE'], label="Training Data")
plt.plot(test_pd['PRICE'], label="Test Data", color='orange')
plt.title("House Price Sales Over Time")
```

```
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.grid()
plt.show()

# Define window for moving averages
window_spec = Window.orderBy("DATE_SOLD").rowsBetween(-30, 0) #
Rolling 30-day window

# Compute Moving Averages for Trendline
train_df = train_df.withColumn("Trend_30",
avg(col("PRICE")).over(window_spec))
test_df = test_df.withColumn("Trend_30",
avg(col("PRICE")).over(window_spec))

# Convert to Pandas for Visualization
train_pd = train_df.select("DATE_SOLD", "PRICE",
"Trend_30").toPandas()
test_pd = test_df.select("DATE_SOLD", "PRICE", "Trend_30").toPandas()

# Set Date as Index
train_pd.set_index('DATE_SOLD', inplace=True)
test_pd.set_index('DATE_SOLD', inplace=True)

# Plot Prices with Trendlines
plt.figure(figsize=(14, 6))
plt.plot(train_pd['PRICE'], label="Training Data", alpha=0.6)
plt.plot(test_pd['PRICE'], label="Test Data", color='orange',
alpha=0.6)

# Add Moving Average Trendlines
plt.plot(train_pd['Trend_30'], label="Training Trend (30-day MA)",
color='blue', linestyle="dashed")
plt.plot(test_pd['Trend_30'], label="Test Trend (30-day MA)",
color='red', linestyle="dashed")
```



```
# Customize Plot
plt.title("House Price Sales Over Time with Trendlines")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.grid()
plt.show()

!pip install sktime

# Convert PySpark DataFrame to Pandas
df_pd = df.select("DATE_SOLD", "PRICE").toPandas()

# Ensure DATE_SOLD is in datetime format
df_pd["DATE_SOLD"] = pd.to_datetime(df_pd["DATE_SOLD"])

# Set DATE_SOLD as index and resample by week ('W')
df_pd.set_index("DATE_SOLD", inplace=True)
y = df_pd["PRICE"].resample("W").mean()

# Plot Time Series
plt.figure(figsize=(15, 6))
plt.plot(y, marker="o", linestyle="-", color="b", label="Avg Weekly Price")
plt.title("Highest Sale with Time", fontsize=20, color="black")
plt.xlabel("Date Sold", fontsize=12)
plt.ylabel("Average Price", fontsize=12)
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.show()
```

```
df_t2.printSchema()

from pyspark.sql.functions import when, year, col
from pyspark.ml.feature import StringIndexer, OneHotEncoder
# Dynamically get the current year
current_year = 2025 # Replace with year(current_date()) if needed

# Create Age Category column
df_t2 = df_t2.withColumn(
    "BUILD_AGE_CAT",
    when((current_year - col("BUILD_YEAR")) > 50, "Old") # Houses
older than 50 years
    .when((current_year - col("BUILD_YEAR")) >= 20, "Moderate") #
Houses between 20-50 years
    .otherwise("New") # Houses less than 20 years old
)

# Label Encoding for Linear Regression & GLR
df_t2 = df_t2.withColumn(
    "BUILD_AGE_LABEL",
    when(col("BUILD_AGE_CAT") == "Old", 2)
    .when(col("BUILD_AGE_CAT") == "Moderate", 1)
    .otherwise(0)
)

# Label Encoding for Random Forest & XGBoost
# Convert categorical labels to indices
indexer = StringIndexer(inputCol="BUILD_AGE_CAT",
outputCol="BUILD_AGE_INDEX")
df_t2 = indexer.fit(df_t2).transform(df_t2)

# Apply One-Hot Encoding (For Linear Regression)
encoder = OneHotEncoder(inputCol="BUILD_AGE_INDEX",
outputCol="AGE_VECTOR")
```

```
df_t2 = encoder.fit(df_t2).transform(df_t2)

# Final Data Structure
df_t2.select("BUILD_YEAR", "BUILD_AGE_CAT", "BUILD_AGE_LABEL",
"AGE_VECTOR").show(5, truncate=False)

df_t2.printSchema()

df_t2 = df_t2.drop("AGE_CATEGORY", "BUILD_AGE_CAT", "DATE_SOLD")

df_t2 = df_t2.dropna()

df_t2.columns

tree_features =
['BEDROOMS', 'BATHROOMS', 'LAND_AREA', 'FLOOR_AREA', 'BUILD_YEAR', 'CBD_DIST',
'NEAREST_STN_DIST', 'LATITUDE', 'LONGITUDE', 'NEAREST_SCH_DIST', 'NEAREST_SCH_RANK',
'YEAR', 'MONTH', 'DAY', 'YEAR_SOLD', 'MONTH_SOLD', 'DAY_SOLD',
'DATE_TIMESTAMP', 'PROPERTY_AGE', 'LAG_1', 'LAG_3', 'MA_3', 'MA_6', 'QUARTER',
'WEEK_OF_YEAR', 'DAY_OF_WEEK', 'BUILD_AGE_LABEL', 'BUILD_AGE_INDEX']

linear_features =
['BEDROOMS', 'BATHROOMS', 'LAND_AREA', 'FLOOR_AREA', 'BUILD_YEAR', 'CBD_DIST',
'NEAREST_STN_DIST', 'LATITUDE', 'LONGITUDE', 'NEAREST_SCH_DIST', 'NEAREST_SCH_RANK',
'YEAR', 'MONTH', 'DAY', 'YEAR_SOLD', 'MONTH_SOLD', 'DAY_SOLD',
'DATE_TIMESTAMP', 'PROPERTY_AGE', 'LAG_1', 'LAG_3', 'MA_3', 'MA_6', 'QUARTER',
'WEEK_OF_YEAR', 'DAY_OF_WEEK', 'BUILD_AGE_INDEX', 'AGE_VECTOR']

# Assemble Features
from pyspark.ml.linalg import Vectors
from pyspark.ml.feature import VectorAssembler

assembler_tree = VectorAssembler(inputCols=tree_features,
outputCol="tree_features")

df_tr = assembler_tree.transform(df_t2)

assembler_linear = VectorAssembler(inputCols=linear_features,
outputCol="linear_features")

df_lr = assembler_linear.transform(df_tr)

target_col = "PRICE"
```

```
# Keep only the "features" vector and the target variable
df_final_tr = df_tr.select("tree_features", target_col)
df_final_tr.show()

# Keep only the "features" vector and the target variable
df_final_lr = df_lr.select("linear_features", target_col)
df_final_lr.show()

#Normalizing the features
# Define StandardScaler
lr_scaler = StandardScaler(inputCol="linear_features",
outputCol="scaledFeaturesLR", withStd=True, withMean=False)
tree_scaler = StandardScaler(inputCol="tree_features",
outputCol="scaledFeaturesTR", withStd=True, withMean=False)

# Fit the scaler to the data
lr_scaler_model = lr_scaler.fit(df_final_lr)
tree_scaler_model = tree_scaler.fit(df_final_tr)

# Transform the data using the fitted scaler
df_linear = lr_scaler_model.transform(df_final_lr)
df_tree = tree_scaler_model.transform(df_final_tr)

# Show transformed data
df_linear.show()
df_tree.show()

# Define the train-test split ratio (80% train, 20% test)
train_ratio = 0.8
test_ratio = 0.2
seed = 42 # Set seed for reproducibility

# Split Linear Model Dataset
```

```

train_linear, test_linear = df_linear.randomSplit([train_ratio,
test_ratio], seed=seed)

# Split Tree Model Dataset

train_tree, test_tree = df_tree.randomSplit([train_ratio, test_ratio],
seed=seed)

# Print the number of rows in each set

print(f"Linear Model: Training Rows = {train_linear.count()}, Test
Rows = {test_linear.count()}")

print(f"Tree Model: Training Rows = {train_tree.count()}, Test Rows =
{test_tree.count()}")

from pyspark.ml.regression import LinearRegression,
RandomForestRegressor, GBRegressor, DecisionTreeRegressor

# Define the four models with their respective feature columns

models = {

    "Linear Regression":
LinearRegression(featuresCol="scaledFeaturesLR", labelCol="PRICE"),

    "Generalized Regression":
GeneralizedLinearRegression(featuresCol="scaledFeaturesLR",
labelCol="PRICE", family="gaussian"),

    "Random Forest":
RandomForestRegressor(featuresCol="scaledFeaturesTR",
labelCol="PRICE", numTrees=100),

    "Gradient Boosting": GBRegressor(featuresCol="scaledFeaturesTR",
labelCol="PRICE"),

    "Decision Tree":
DecisionTreeRegressor(featuresCol="scaledFeaturesTR",
labelCol="PRICE")

}

# Dictionary to store trained models

trained_models = {}

# Mapping models to their respective training datasets

```

```
dataset_map = {
    "Linear Regression": train_linear, # Use train_linear for Linear
    Regression
    "Generalized Regression": train_linear, # Use train_linear for
    GLR
    "Random Forest": train_tree,          # Use train_tree for Tree-
    based models
    "Gradient Boosting": train_tree,
    "Decision Tree": train_tree
}

# Iterate through each model and fit it to the respective dataset
for model_name, model in models.items():
    print(f"Training {model_name}...")

    # Get the appropriate dataset
    train_data = dataset_map[model_name]

    # Train the model
    trained_model = model.fit(train_data)

    # Store the trained model
    trained_models[model_name] = trained_model

print("Training complete!")

# Dictionary to store predictions
predictions = {}
y_pred = {} # Dictionary to store only the predicted values
separately

# Generate predictions for each trained model
for model_name, trained_model in trained_models.items():
    print(f"Generating predictions for {model_name}...")
```

```

# Select the appropriate test dataset
test_data = dataset_map[model_name] # Uses the dataset mapping
from training

# Make predictions
predictions[model_name] = trained_model.transform(test_data)

# Store only the predicted values separately
y_pred[model_name] = predictions[model_name].select("prediction")

print("Predictions complete!")

# Define evaluators
rmse_evaluator = RegressionEvaluator(labelCol="PRICE",
predictionCol="prediction", metricName="rmse")
mae_evaluator = RegressionEvaluator(labelCol="PRICE",
predictionCol="prediction", metricName="mae")
r2_evaluator = RegressionEvaluator(labelCol="PRICE",
predictionCol="prediction", metricName="r2")

# Store evaluation results
evaluation_results = {}

# Evaluate each model
for model_name, preds in predictions.items():
    rmse = rmse_evaluator.evaluate(preds)
    mae = mae_evaluator.evaluate(preds)
    r2 = r2_evaluator.evaluate(preds)

    evaluation_results[model_name] = {"RMSE": rmse, "MAE": mae, "R2":
r2}

```



```
print(f"{model_name}: RMSE={rmse:.3f}, MAE={mae:.3f},  
R²={r2:.3f}")
```

```
# Extract evaluation results
```

```
models = list(evaluation_results.keys())
```

```
rmse_scores = [evaluation_results[m]["RMSE"] for m in models]
```

```
r2_scores = [evaluation_results[m]["R2"] for m in models]
```

```
# Set positions for bars
```

```
x = np.arange(len(models))
```

```
# Plot RMSE
```

```
plt.figure(figsize=(5, 3))
```

```
plt.bar(x, rmse_scores, color='red', alpha=0.7)
```

```
plt.xticks(x, models, rotation=45)
```

```
plt.ylabel("RMSE (Lower is Better)")
```

```
plt.title("Model Comparison - RMSE")
```

```
plt.show()
```

```
# Plot R² Score
```

```
plt.figure(figsize=(5, 3))
```

```
plt.bar(x, r2_scores, color='blue', alpha=0.7)
```

```
plt.xticks(x, models, rotation=45)
```

```
plt.ylabel("R² Score (Higher is Better)")
```

```
plt.title("Model Comparison - R² Score")
```

```
plt.show()
```

```
# Plot MAE
```

```
plt.figure(figsize=(5, 3))
```

```
mae_scores = [evaluation_results[m]["MAE"] for m in models]
```

```
plt.bar(x, mae_scores, color='green', alpha=0.7)
```

```
plt.xticks(x, models, rotation=45)
```

```
plt.ylabel("MAE (Lower is Better)")
```

```
plt.title("Model Comparison - MAE")
plt.show()

# Select model with highest R²
best_model_name = max(evaluation_results, key=lambda m:
evaluation_results[m]["R2"])
print(f"Best model is: {best_model_name}")

# Select a model (example: Random Forest)
selected_model = trained_models["Gradient Boosting"]
predictions = selected_model.transform(test_tree).select("PRICE",
"prediction").toPandas()

# Scatter plot
plt.figure(figsize=(8, 6))
sns.scatterplot(x=predictions["PRICE"], y=predictions["prediction"],
alpha=0.5)
plt.plot([predictions["PRICE"].min(), predictions["PRICE"].max()],
[predictions["PRICE"].min(), predictions["PRICE"].max()],
color="red", linestyle="--") # Ideal line
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted House Prices")
plt.show()

# Calculate residuals
predictions["Residuals"] = predictions["PRICE"] -
predictions["prediction"]

# Residuals plot
plt.figure(figsize=(8, 6))
sns.histplot(predictions["Residuals"], bins=30, kde=True)
plt.xlabel("Residuals")
plt.ylabel("Frequency")
```

```
plt.title("Residuals Distribution")
plt.show()

# Select a trained model (Example: Random Forest)
selected_model = trained_models["Gradient Boosting"]

# Make predictions
predictions_df = selected_model.transform(test_tree).select("PRICE",
"prediction").toPandas()

# Compute Residuals (Error = Actual - Predicted)
predictions_df["Residuals"] = predictions_df["PRICE"] -
predictions_df["prediction"]

# Scatter Plot: Residuals vs Predicted
plt.figure(figsize=(8, 6))
sns.scatterplot(x=predictions_df["prediction"],
y=predictions_df["Residuals"], alpha=0.5)
plt.axhline(y=0, color="red", linestyle="--") # Reference line at 0
plt.xlabel("Predicted Prices")
plt.ylabel("Residuals (Actual - Predicted)")
plt.title("Residuals Scatter Plot")
plt.show()

# Choose the model name (update as needed)
selected_model = "Gradient Boosting" # Change this to the desired
model name

# Extract the Spark DataFrame containing predictions for the selected
model
y_pred_df = y_pred[selected_model] # Get predictions for the chosen
model

# Convert Spark DataFrame predictions to a NumPy array
```

```
y_pred_np = np.array(y_pred_df.select("prediction").rdd.flatMap(lambda
x: x).collect())
```

```
# Ensure it has the correct shape
```

```
y_pred_np = y_pred_np.reshape(-1, 1) # Reshape to 2D for OLS
```

```
print(y_pred_np[:5]) # Display first 5 predictions
```

```
# Select the model you want to analyze
```

```
selected_model = "Gradient Boosting" # Change to desired model name
```

```
# Extract y_pred DataFrame from dictionary
```

```
y_pred_df = y_pred[selected_model] # Get the Spark DataFrame for
predictions
```

```
# Convert Spark DataFrame column "prediction" to a NumPy array
```

```
y_pred_np = np.array(y_pred_df.select("prediction").rdd.flatMap(lambda
x: x).collect())
```

```
print(y_pred_np[:5]) # Preview first few values
```

```
# Extract features from the same test set used for predictions
```

```
X_np = np.array(test_data.select("scaledFeaturesTR").rdd.map(lambda x:
x[0]).collect())
```

```
# Ensure y_pred is extracted from the same dataset
```

```
y_pred_np = np.array(y_pred_df.select("prediction").rdd.flatMap(lambda
x: x).collect())
```

```
print("Fixed Shape of X_np:", X_np.shape)
```

```
print("Fixed Shape of y_pred_np:", y_pred_np.shape)
```

```
import statsmodels.api as sm # Import statsmodels
```

```
X_np = sm.add_constant(X_np) # Add intercept term to feature matrix
```

```

model = sm.OLS(y_pred_np, X_np).fit()
print(model.summary()) # Display OLS regression results

# Compute Cook's Distance
influence = model.get_influence()
cooks_d, _ = influence.cooks_distance # Extract Cook's distances

# Plot Cook's Distance
plt.figure(figsize=(10, 5))
plt.stem(np.arange(len(cooks_d)), cooks_d, markerfmt=",")
plt.axhline(4 / len(X_np), color="r", linestyle="--", label="Threshold (4/n)")
plt.title("Cook's Distance Plot")
plt.xlabel("Observation Index")
plt.ylabel("Cook's Distance")
plt.legend()
plt.show()

# Identify high influence points
threshold = 50 / len(X_np)
high_influence_points = np.where(cooks_d > threshold)[0]
print(f"High Influence Points (indices): {high_influence_points}")

# Convert the PySpark DataFrame to a pandas DataFrame
test_dataset_pd = test_tree.toPandas()

# Extract features and target as NumPy arrays
X = np.array(test_dataset_pd['scaledFeaturesTR'].tolist())
y = test_dataset_pd['PRICE'].values # Assuming 'PRICE' is your target column

# Define model with pipeline
model = make_pipeline(StandardScaler(), Ridge(alpha=1.0))

```

```
# Compute learning curve
train_sizes, train_scores, val_scores = learning_curve(
    model, X, y, cv=5, scoring="neg_mean_squared_error",
    train_sizes=np.linspace(0.1, 1.0, 10))

# Convert negative MSE to positive RMSE
train_scores = np.sqrt(-train_scores)
val_scores = np.sqrt(-val_scores)

# Plot learning curve
plt.figure(figsize=(8, 5))
plt.plot(train_sizes, train_scores.mean(axis=1), label="Training",
marker="o")
plt.plot(train_sizes, val_scores.mean(axis=1), label="Validation",
marker="s")
plt.xlabel("Training Set Size")
plt.ylabel("Root Mean Squared Error (RMSE)")
plt.title("Learning Curve")
plt.legend()
plt.show()
```